

**Faculdade de Informática e Administração Paulista (FIAP)**  
**Engenharia de Software – 3ESPW**

Camila Pedroza da Cunha – RM 558768  
Isabelle Dallabeneta Carlesso – RM 554592  
Nicolli Amy Kassa – RM 559104  
Pedro Almeida e Camacho – RM 556831  
Renan Dias Utida – RM 558540

**Sprint 1 - FORD**  
**CyberSecurity**

**São Paulo, SP**  
**2026**

# SpecRadar - API

Link do Repositório: <https://github.com/renan-utida/Sprint-Ford-API>

## 1. Segurança de Entrada e Validação de Dados

### Validação de entradas do usuário

Sanitização contra SQL Injection, XSS, command injection e entradas malformadas.

Todas as entradas da API são validadas via **Bean Validation** antes de chegarem à camada de serviço. A sanitização contra SQL Injection é garantida pelo Spring Data JPA, que utiliza queries parametrizadas, nunca há concatenação direta de strings na construção de queries SQL. XSS e command injection não são vetores aplicáveis pois a API produz e consome exclusivamente JSON, sem renderização de HTML.

O tratamento de entradas malformadas é feito pelo **GlobalExceptionHandler**, que intercepta **HttpMessageNotReadableException** e retorna uma resposta padronizada sem expor detalhes internos.

#### Prints:

Código do **GlobalExceptionHandler.java**, especificamente o método **handleJsonInvalido** que trata **HttpMessageNotReadableException** e um teste no Insomnia enviando uma marca inválida ("MARCA\_INVALIDA") e mostrando o response.

The image shows a development environment with two main parts: a REST client (Insomnia) on the left and a code editor on the right.

**Left Panel (Insomnia):**

- Request:** A POST request to `__base_url_/api/veiculos`. The body is a JSON object: `{ "marca": "MARCA_INVALIDA", "modelo": "F40", "versao": "Base", "ano": 2025 }`. It is highlighted with a red box.
- Response:** A 400 Bad Request response. The body is a JSON object: `{ "status": 400, "erro": "Erro de validação", "mensagem": "Valor inválido para o campo 'marca': 'MARCA_INVALIDA'", "path": "/api/veiculos", "timestamp": "2026-05-19T00:59:36.9752873", "campos": null }`. It is also highlighted with a red box.
- Metadata:** Shows "400 Bad Request", "37 ms", "190 B", and "17 Minutes Ago".

**Right Panel (Code Editor):**

- File:** `GlobalExceptionHandler.java`.
- Code:** Shows the `handleJsonInvalido` method. It catches a `HttpMessageNotReadableException` and returns a `ResponseEntity` with status `HttpStatus.BAD_REQUEST` and a body containing an error message and the request URI. The code is as follows:

```
24 public class GlobalExceptionHandler {
72
73     // 400 - Enum invalido ou JSON malformado
74     @ExceptionHandler({HttpMessageNotReadableException.class}) no usages 2 Renan
75     @ public ResponseEntity<ErroResponse> handleJsonInvalido(
76         HttpMessageNotReadableException ex,
77         HttpServletRequest request
78     ) {
79         String mensagem = "Requisição inválida";
80
81         if (ex.getCause() instanceof InvalidFormatException ife) {
82             if (ife.getTargetType() != null && ife.getTargetType().isEnum()) {
83                 String valorInvalido = ife.getValue().toString();
84                 String caminho = ife.getPath().isEmpty() ? "campo"
85                     : ife.getPath().getFirst().getFieldName();
86                 mensagem = "Valor inválido para o campo " + caminho
87                     + ": " + valorInvalido + " ";
88             }
89         }
90
91         return ResponseEntity
92             .status(HttpStatus.BAD_REQUEST)
93             .body(ErroResponse.of(
94                 HttpStatus.BAD_REQUEST.value(),
95                 erro: "Erro de validação",
96                 mensagem,
97                 request.getRequestURI()
98             ));
99     }
100 }
```

H2 Console mostrando a tabela **ford\_veiculos** com dados reais para evidenciar que não há injeção SQL (os dados estão limpos e estruturados).

Run

Run Selected

Auto complete

Clear

SQL statement:

SELECT \* FROM ford\_veiculos;

SELECT \* FROM ford\_veiculos;

| ID | MARCA      | MODELO | VERSAO       | ANO  | ATIVO | CRIADO_EM                  |
|----|------------|--------|--------------|------|-------|----------------------------|
| 1  | TOYOTA     | Hilux  | GR-Sport     | 2025 | 1     | 2026-05-19 00:54:30.360152 |
| 2  | VOLKSWAGEN | Amarok | V6 Extreme   | 2025 | 1     | 2026-05-19 00:54:30.360152 |
| 3  | CHEVROLET  | S10    | High Country | 2025 | 1     | 2026-05-19 00:54:30.360152 |
| 4  | FORD       | Ranger | Raptor       | 2025 | 1     | 2026-05-19 00:54:30.372161 |

(4 rows, 4 ms)

## Normalização e validação de parâmetros de API

Garantir que marca, modelo, versão e atributos sigam padrões definidos.

A normalização dos parâmetros de API é garantida pelo uso de **enums** para os campos **marca** e **role**. O enum **MarcaVeiculo** define as únicas marcas aceitas pelo sistema, onde qualquer valor fora desse conjunto é automaticamente rejeitado com HTTP 400, sem necessidade de código de validação adicional. A configuração **accept-case-insensitive-enums=true** no **application.properties** garante que variações de capitalização como **"toyota"**, **"Toyota"** e **"TOYOTA"** são todas normalizadas para o valor **TOYOTA**, eliminando inconsistências de entrada.

Os campos **modelo**, **versao** e **atributo** seguem padrões definidos via Bean Validation com **@NotBlank** e **@Size**, garantindo que nunca chegam vazios ou com tamanho fora dos limites ao service.

**Prints:**

Código do **MarcaVeiculo.java** mostrando o enum completo com todos os valores.

```
1 package br.com.ford.speccradar.domain.enums;
2
3 public enum MarcaVeiculo { @ Renan
4     FORD, no usages
5     TOYOTA,
6     VOLKSWAGEN, 6 usages
7     CHEVROLET, no usages
8     FIAT, no usages
9     HYUNDAI, no usages
10    NISSAN, no usages
11    MITSUBISHI, no usages
12    JEEP, 4 usages
13    RAM, no usages
14    MERCEDES no usages
15 }
```

Trecho do **application.properties** destacando a configuração mencionada.

```
application.properties x
24 cors.allowed-origins=${UUKS_ALLOWED_ORIGINS:http://localhost:3000,http://localhost:8081}
25
26 # Jackson
27 # Desabilitando o case-sensitive para aceitar qualquer capitalizacao
28 spring.jackson.serialization.fail-on-unknown-properties=false
29 spring.jackson.mapper.accept-case-insensitive-enums=true
30
```

Teste no Insomnia digitando **"toyota"** (minúsculo) no campo marca e mostrando que é aceito normalmente (evidência da normalização case-insensitive).

POST Cadastrar veículo x +

POST ▾ `_.base_url /api/veiculos` Send ▾ 201 Created 1.51 s 127 B

Params Body Auth Headers 3 Scripts Docs Preview Headers 12 Cookies Tests 0/0 → Mock Con

JSON ▾ Preview ▾

```
1 {
2   "marca": "toyota",
3   "modelo": "Hilux",
4   "versao": "GR-Sport",
5   "ano": 2025
6 }
```

```
1 {
2   "id": 5,
3   "marca": "TOYOTA",
4   "modelo": "Hilux",
5   "versao": "GR-Sport",
6   "ano": 2025,
7   "ativo": true,
8   "criadoEm": "2026-05-19T01:56:41.9425103"
9 }
```

Código do **VeiculoRequest.java** mostrando as anotações **@NotBlank** e **@Size** nos campos **modelo**, **versao** e do **EspecificacaoRequest.java** no campo **atributo**.

```
VeiculoRequest.java x
1 package br.com.ford.specradar.dto.request;
2
3 import br.com.ford.specradar.domain.enums.MarcaVeiculo;
4 import jakarta.validation.constraints.Max;
5 import jakarta.validation.constraints.Min;
6 import jakarta.validation.constraints.NotBlank;
7 import jakarta.validation.constraints.NotNull;
8 import jakarta.validation.constraints.Size;
9 import lombok.Data;
10
11 @Data 9 usages 2 Renan
12 public class VeiculoRequest {
13
14     @NotNull(message = "Marca é obrigatória")
15     private MarcaVeiculo marca;
16
17     @NotBlank(message = "Modelo é obrigatório")
18     @Size(min = 1, max = 100, message = "Modelo deve ter no máximo 100 caracteres")
19     private String modelo;
20
21     @NotBlank(message = "Versão é obrigatória")
22     @Size(min = 1, max = 100, message = "Versão deve ter no máximo 100 caracteres")
23     private String versao;
24
25     @NotNull(message = "Ano é obrigatório")
26     @Min(value = 1900, message = "Ano inválido")
27     @Max(value = 2100, message = "Ano inválido")
28     private Integer ano;
29 }

EspecificacaoRequest.java x
1 package br.com.ford.specradar.dto.request;
2
3 import jakarta.validation.constraints.NotBlank;
4 import jakarta.validation.constraints.NotNull;
5 import jakarta.validation.constraints.Size;
6 import lombok.Data;
7
8 @Data 9 usages 2 Renan
9 public class EspecificacaoRequest {
10
11     @NotBlank(message = "Atributo é obrigatório")
12     @Size(min = 1, max = 100, message = "Atributo deve ter no máximo 100 caracteres")
13     private String atributo;
14
15     @NotBlank(message = "Valor é obrigatório")
16     @Size(min = 1, max = 255, message = "Valor deve ter no máximo 255 caracteres")
17     private String valor;
18
19     @Size(max = 50, message = "Unidade deve ter no máximo 50 caracteres")
20     private String unidade;
21
22     @NotNull(message = "Disponível é obrigatório")
23     private Boolean disponivel;
24 }
```

## Limitação de tamanho e formato de entrada

Prevenir ataques de payload flooding e buffer overflow.

Todos os campos String da API possuem limite máximo de tamanho definido via **@Size**, impedindo que payloads excessivamente grandes cheguem à camada de serviço ou ao banco de dados. Campos numéricos como **ano** possuem validação de intervalo via **@Min** e **@Max**, prevenindo valores absurdos. Campos obrigatórios são marcados com **@NotBlank** ou **@NotNull**, impedindo entradas vazias ou nulas.

O rate limiting implementado via **RateLimitFilter** com Bucket4j atua como segunda camada de proteção contra payload flooding, um IP que tente enviar requisições em volume excessivo é bloqueado com HTTP 429 após 20 requisições por minuto, antes mesmo de qualquer processamento de dados.

### Prints:

Código do **VeiculoRequest.java** completo mostrando todas as anotações de validação (**@NotNull**, **@NotBlank**, **@Size**, **@Min**, **@Max**) e do **EspecificacaoRequest.java** mostrando os **@Size** nos campos **atributo** (max 100) e **valor** (max 255).

```
VeiculoRequest.java
1 package br.com.ford.specradar.dto.request;
2
3 import br.com.ford.specradar.domain.enums.MarcaVeiculo;
4 import jakarta.validation.constraints.Max;
5 import jakarta.validation.constraints.Min;
6 import jakarta.validation.constraints.NotBlank;
7 import jakarta.validation.constraints.NotNull;
8 import jakarta.validation.constraints.Size;
9 import lombok.Data;
10
11 @Data
12 public class VeiculoRequest {
13
14     @NotNull(message = "Marca é obrigatória")
15     private MarcaVeiculo marca;
16
17     @NotBlank(message = "Modelo é obrigatório")
18     @Size(min = 1, max = 100, message = "Modelo deve ter no máximo 100 caracteres")
19     private String modelo;
20
21     @NotBlank(message = "Versão é obrigatória")
22     @Size(min = 1, max = 100, message = "Versão deve ter no máximo 100 caracteres")
23     private String versao;
24
25     @NotNull(message = "Ano é obrigatório")
26     @Min(value = 1900, message = "Ano inválido")
27     @Max(value = 2100, message = "Ano inválido")
28     private Integer ano;
29 }
```

```
EspecificacaoRequest.java
1 package br.com.ford.specradar.dto.request;
2
3 import jakarta.validation.constraints.NotBlank;
4 import jakarta.validation.constraints.NotNull;
5 import jakarta.validation.constraints.Size;
6 import lombok.Data;
7
8 @Data
9 public class EspecificacaoRequest {
10
11     @NotBlank(message = "Atributo é obrigatório")
12     @Size(min = 1, max = 100, message = "Atributo deve ter no máximo 100 caracteres")
13     private String atributo;
14
15     @NotBlank(message = "Valor é obrigatório")
16     @Size(min = 1, max = 255, message = "Valor deve ter no máximo 255 caracteres")
17     private String valor;
18
19     @Size(max = 50, message = "Unidade deve ter no máximo 50 caracteres")
20     private String unidade;
21
22     @NotNull(message = "Disponível é obrigatório")
23     private Boolean disponivel;
24 }
```

Código do **UsuarioRequest.java** mostrando os **@Size** em **nome** (max 100), **email** (max 150) e **senha** (max 100).

```
1 package br.com.ford.specradar.dto.request;
2
3 import br.com.ford.specradar.domain.enums.RoleUsuario;
4 import jakarta.validation.constraints.Email;
5 import jakarta.validation.constraints.NotBlank;
6 import jakarta.validation.constraints.NotNull;
7 import jakarta.validation.constraints.Size;
8 import lombok.Data;
9
10 @Data
11 public class UsuarioRequest {
12
13     @NotBlank(message = "Nome é obrigatório")
14     @Size(min = 2, max = 100, message = "Nome deve ter entre 2 e 100 caracteres")
15     private String nome;
16
17     @NotBlank(message = "Email é obrigatório")
18     @Email(message = "Email inválido")
19     @Size(max = 150, message = "Email deve ter no máximo 150 caracteres")
20     private String email;
21
22     @NotBlank(message = "Senha é obrigatória")
23     @Size(min = 6, max = 100, message = "Senha deve ter entre 6 e 100 caracteres")
24     private String senha;
25
26     @NotNull(message = "Role é obrigatória")
27     private RoleUsuario role;
28 }
```

Teste no Insomnia enviando **modelo** com mais de 100 caracteres e mostrando o response 400.

The screenshot shows an Insomnia API client interface. The request is a POST to `./base_url /api/veiculos`. The body is JSON with the following content:

```
1 {
2   "marca": "TOYOTA",
3   "modelo": "EsteModeloTemMuitoMaisDeUmCentoDeCaracteresParaTestarOLimiteDeValidacaoDoCampoModeloNaApiSpecRadarPa
4   raSprintDaFord",
5   "versao": "GR-Sport",
6   "ano": 2025
7 }
```

The response is a 400 Bad Request with the following JSON body:

```
1 {
2   "status": 400,
3   "erro": "Erro de validação",
4   "mensagem": "Um ou mais campos são inválidos",
5   "path": "/api/veiculos",
6   "timestamp": "2026-05-19T02:28:55.3940811",
7   "campos": {
8     "modelo": "Modelo deve ter no máximo 100 caracteres"
9   }
10 }
```

Teste enviando **ano** com valor **2300** e mostrando o response 400 com campo **ano** inválido.

The screenshot shows an Insomnia API client interface. The request is a POST to `./base_url /api/veiculos`. The body is JSON with the following content:

```
1 {
2   "marca": "TOYOTA",
3   "modelo": "Hilux",
4   "versao": "GR-Sport",
5   "ano": 2300
6 }
```

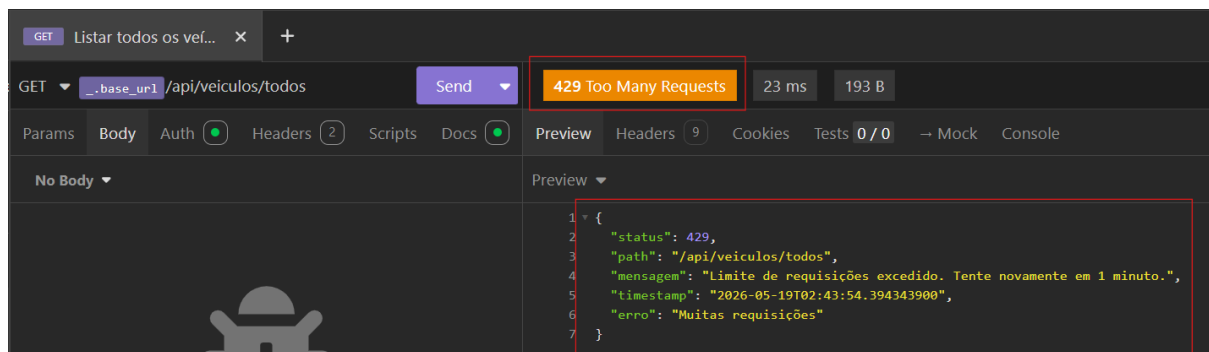
The response is a 400 Bad Request with the following JSON body:

```
1 {
2   "status": 400,
3   "erro": "Erro de validação",
4   "mensagem": "Um ou mais campos são inválidos",
5   "path": "/api/veiculos",
6   "timestamp": "2026-05-19T02:31:45.1335411",
7   "campos": {
8     "ano": "Ano inválido"
9   }
10 }
```

Código do **RateLimitFilter.java** destacando a constante **MAX\_REQUESTS\_PER\_MINUTE = 20** e o bloco com response 429 com Json padronizado.

```
RateLimitFilter.java x
23  /**
24  * Filtro de rate limiting por IP usando Bucket4j.
25  *
26  * Limites aplicados a TODOS os endpoints da aplicação:
27  * - 20 requisições por minuto por IP (janela deslizante)
28  * - IPs que excedem o limite recebem 429 Too Many Requests
29  * - Tentativas bloqueadas são logadas com IP para monitoramento
30  *
31  * Aplicado globalmente para prevenir brute force, scraping e DoS
32  * em qualquer endpoint, incluindo login e Swagger.
33  */
34  @Component & Renan
35  public class RateLimitFilter extends OncePerRequestFilter {
36
37      private static final Logger log = LoggerFactory.getLogger(RateLimitFilter.class); 1 usage
38
39      // Mapa de buckets por IP - cada IP tem seu próprio contador
40      private final Map<String, Bucket> buckets = new ConcurrentHashMap<>(); 1 usage
41
42      // Limite: 20 requisições por minuto por IP
43      private static final int MAX_REQUESTS_PER_MINUTE = 20; 2 usages
44
45      @Override no usages & Renan
46      protected void doFilterInternal(
47          @NonNull HttpServletRequest request,
48          @NonNull HttpServletResponse response,
49          @NonNull FilterChain filterChain
50      ) throws ServletException, IOException {
51
52          String ip = extrairIp(request);
53          Bucket bucket = buckets.computeIfAbsent(ip, this::criarNovoBucket);
54
55          // Tenta consumir 1 token - se houver saldo, deixa passar
56          if (bucket.tryConsume( numTokens: 1)) {
57              filterChain.doFilter(request, response);
58              return;
59          }
60
61          // Sem saldo - bloqueia e loga
62          log.warn("[SEGURANÇA] Rate limit excedido - IP: {} endpoint: {} método: {}",
63              ip, request.getRequestURI(), request.getMethod());
64
65          response.setStatus(HttpStatus.TOO_MANY_REQUESTS.value());
66          response.setContentType("application/json;charset=UTF-8");
67
68          Map<String, Object> body = Map.of(
69              k1: "status", v1: 429,
70              k2: "erro", v2: "Muitas requisições",
71              k3: "mensagem", v3: "Limite de requisições excedido. Tente novamente em 1 minuto.",
72              k4: "path", request.getRequestURI(),
73              k5: "timestamp", LocalDateTime.now().toString()
74          );
75
76          new ObjectMapper().writeValue(response.getOutputStream(), body);
77      }
```

Teste fazendo mais de 20 requisições seguidas e mostrando o response 429.



## Tratamento seguro de erros

Mensagens não podem revelar stack trace, estrutura interna ou tecnologia usada.

O **GlobalExceptionHandler** centraliza o tratamento de todas as exceções da aplicação via **@RestControllerAdvice**. Nenhuma exceção chega ao cliente sem passar por esse handler, e isso garante que stack traces, nomes de classes internas, tecnologias utilizadas (Spring, Hibernate, Oracle) ou detalhes de queries nunca são expostos nas respostas da API.

Cada tipo de erro possui um handler específico que retorna sempre o mesmo formato JSON padronizado com **status**, **erro**, **mensagem**, **path** e **timestamp**. O handler genérico para **Exception.class** é a última barreira, que captura qualquer erro inesperado, loga internamente via SLF4J com a mensagem real para rastreamento, e retorna ao cliente apenas **"Ocorreu um erro inesperado. Tente novamente mais tarde."** sem nenhum detalhe interno.

### Prints:

Código do **GlobalExceptionHandler.java** mostrando o handler genérico **handleGenerico** com o **log.error** interno e o response.

```
GlobalExceptionHandler.java x
24 public class GlobalExceptionHandler {
146
147     // 500 - Erro genérico
148     @ExceptionHandler(Exception.class) 1 usage  Renan
149     @ public ResponseEntity<ErroResponse> handleGenerico(
150         Exception ex,
151         HttpServletRequest request
152     ) {
153         log.error("[ERRO INTERNO] {} : {}", ex.getClass().getName(), ex.getMessage());
154
155         return ResponseEntity
156             .status(HttpStatus.INTERNAL_SERVER_ERROR)
157             .body(ErroResponse.of(
158                 HttpStatus.INTERNAL_SERVER_ERROR.value(),
159                 erro: "Erro interno do servidor",
160                 mensagem: "Ocorreu um erro inesperado. Tente novamente mais tarde.",
161                 request.getRequestURI()
162             ));
163     }
```

Código mostrando o **AuthenticationEntryPoint** e o **AccessDeniedHandler** no **SecurityConfig.java** (evidencia que até os erros de autenticação retornam JSON padronizado sem tecnologia exposta).

```
SecurityConfig.java x
35 public class SecurityConfig {
117
118     @Bean & Renan
119     public AuthenticationEntryPoint authenticationEntryPoint() {
120         return ( HttpServletRequest request, HttpServletResponse response, AuthenticationException authException) -> {
121             log.warn("[SEGURANÇA] Tentativa não autenticada em {} - IP: {}",
122                     request.getRequestURI(), request.getRemoteAddr());
123
124             response.setStatus(401);
125             response.setContentType("application/json;charset=UTF-8");
126             Map<String, Object> body = Map.of(
127                 k1: "status", v1: 401,
128                 k2: "erro", v2: "Não autenticado",
129                 k3: "mensagem", v3: "É necessário realizar o login para acessar este recurso",
130                 k4: "path", request.getRequestURI(),
131                 k5: "timestamp", LocalDateTime.now().toString()
132             );
133             new ObjectMapper().writeValue(response.getOutputStream(), body);
134         };
135
136     @Bean & Renan
137     public AccessDeniedHandler accessDeniedHandler() {
138         return ( HttpServletRequest request, HttpServletResponse response, AccessDeniedException accessDeniedException) -> {
139             log.warn("[SEGURANÇA] Acesso negado em {} - IP: {}",
140                     request.getRequestURI(), request.getRemoteAddr());
141
142             response.setStatus(403);
143             response.setContentType("application/json;charset=UTF-8");
144             Map<String, Object> body = Map.of(
145                 k1: "status", v1: 403,
146                 k2: "erro", v2: "Acesso negado",
147                 k3: "mensagem", v3: "Você não tem permissão para acessar este recurso",
148                 k4: "path", request.getRequestURI(),
149                 k5: "timestamp", LocalDateTime.now().toString()
150             );
151             new ObjectMapper().writeValue(response.getOutputStream(), body);
152         };
153     }
}
```

Teste no Insomnia tentando acessar qualquer endpoint sem token e mostrando o response 401 limpo.

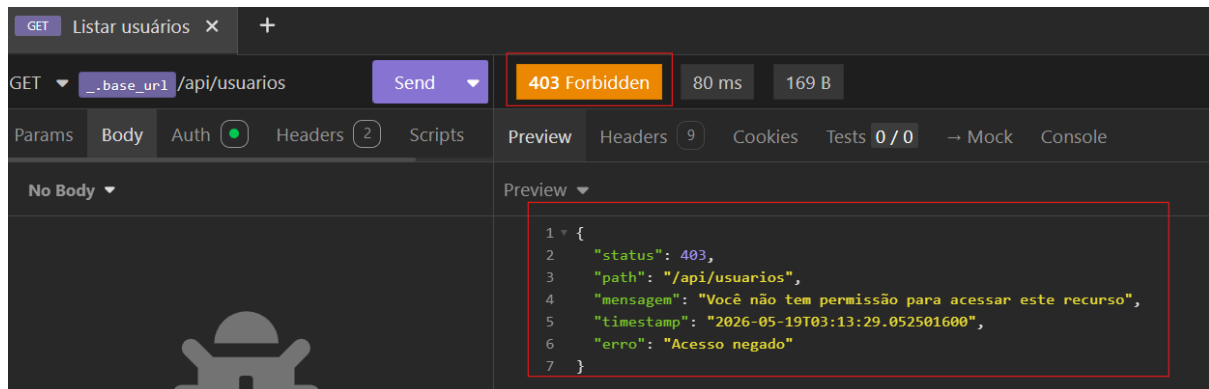
The screenshot shows the Insomnia API client interface. At the top, a GET request is defined for the endpoint `base_url /api/veiculos/todos`. The response status is **401 Unauthorized**, with a response time of 78 ms and a body size of 184 B. The response body is a JSON object:

```
{
  "status": 401,
  "path": "/api/veiculos/todos",
  "mensagem": "É necessário realizar o login para acessar este recurso",
  "timestamp": "2026-05-19T03:07:39.425224200",
  "erro": "Não autenticado"
}
```

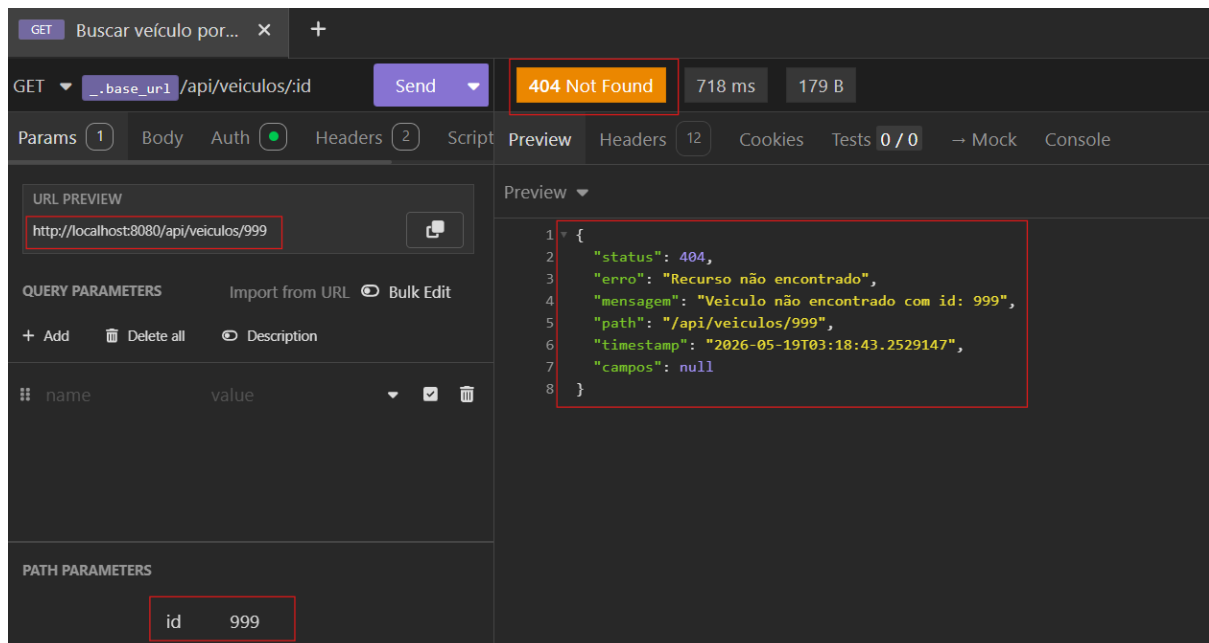
The interface also shows the 'OpenAPI' section with the following configuration:

```
{
  "scheme": "http",
  "base_path": "",
  "host": "localhost:8080",
  "bearerToken": ""
}
```

Teste com ANALISTA tentando acessar endpoint exclusivo de ADMIN e mostrando o response 403 limpo.



Teste buscando recurso inexistente (ex: **GET /api/veiculos/999**) mostrando o response 404 sem stack trace.



## 2. Autenticação e Autorização

### Implementação de autenticação segura (JWT, OAuth2, etc.)

Tokens com expiração, assinatura forte e renovação controlada.

A autenticação é implementada via **JWT (JSON Web Token)** com algoritmo **HS256**. O secret é gerado com `openssl rand -base64 64`, produzindo uma chave de 512 bits, bem acima do mínimo de 256 bits exigido pelo algoritmo HS256. A chave nunca é hardcoded no código, aonde é lida exclusivamente via variável de ambiente **JWT\_SECRET** no arquivo `.env`, que está no `.gitignore` e nunca vai para o repositório.

Cada token carrega três informações no payload: o **subject** com o email do usuário, o **role** para controle de acesso, e o **nome** para exibição. A expiração padrão é de **8 horas** (28.800.000 ms), configurável via variável de ambiente **JWT\_EXPIRATION**. Tokens expirados ou com assinatura incorreta são rejeitados pelo **JwtFilter** com HTTP 401, sem revelar o motivo específico ao cliente.

O fluxo completo de autenticação é: o cliente envia email e senha para **POST /api/auth/login**, o **AuthenticationManager** valida as credenciais contra o banco via BCrypt, e em caso de sucesso o **JwtService** gera e retorna o token com **tipo**, **token** e **expiraEm**.

### Prints:

Código do **JwtService.java** mostrando o método **gerarToken** com os claims de **role** e **nome**, o **subject**, a **issuedAt**, a **expiration** e o **signWith**.

```
17
18 @Service & campz Cunha +1
19 public class JwtService {
20
21     @Value("${jwt.secret}")
22     private String secret;
23
24     @Value("${jwt.expiration}")
25     private long expiration;
26
27     // Gerar token
28
29     @
30     public String gerarToken(Usuario usuario) { 13 usages & campz Cunha
31         Map<String, Object> claims = new HashMap<>();
32         claims.put("role", usuario.getRole().name());
33         claims.put("nome", usuario.getNome());
34
35         return Jwts.builder()
36             .claims(claims)
37             .subject(usuario.getEmail())
38             .issuedAt(new Date())
39             .expiration(new Date(System.currentTimeMillis() + expiration))
40             .signWith(getSecretKey())
41             .compact();
42
43     // Extrair dados do token
```

Código do **JwtService.java** mostrando o método **getSecretKey** evidenciando que a chave vem da variável de ambiente e usa **Keys.hmacShaKeyFor**.

```
19 public class JwtService {
20
21     @
22     private SecretKey getSecretKey() { 2 usages & campz Cunha
23         return Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
24     }
25 }
```

Trecho do **.env.example** mostrando as variáveis **JWT\_SECRET** e **JWT\_EXPIRATION**, comprova que o secret nunca é hardcoded.

```
.env.example x
1  # Projeto SpecRadar - Variáveis de Ambiente
2  # Copie este arquivo para .env e preencha os valores
3  # cp .env.example .env
4
5  SPRING_PROFILE=dev
6
7  # Oracle FIAP
8  ORACLE_URL=jdbc:oracle:thin:@oracle.fiap.com.br:1521
9  ORACLE_USER=seu_rm_aqui
10 ORACLE_PASSWORD=sua_senha_aqui
11
12 # JWT - gere com: openssl rand -base64 64
13 JWT_SECRET=
14 JWT_EXPIRATION=28800000
```

Teste no Insomnia fazendo login com **POST /api/auth/login** e mostrando o response com o token.

Insomnia interface showing a successful POST request to `/api/auth/login`. The status is **200 OK** with a response time of 1.12 s and 320 B. The response body is a JSON object:

```
{
  "token": "eyJhbGciOiJIUzUxMiJ9.eyJyb2xlIjoiaXQ5bTE1VEEiLCJub211IjoiaXQ5bG1zdGEgRm9yZCIsInN1YiI6ImFuYVwpc3RhQHhNwZWNyYWRhcy5jb20iLCJpYXQiOiE3NzkzM1M0M0ImV4cCI6MTc3OTQwMjM0M0.18HwFRAZNSbo1Q9owdftT4ZxblcbyuIqcEOW0GFCVxca57r9xuJQ0yZy-SIMUHCytrGk0PD1V9dboeJnIY6DQ",
  "tipo": "Bearer",
  "expiraEm": "2026-05-21T19:25:03.1215071"
}
```

Teste enviando um token inválido ou expirado em qualquer endpoint protegido e mostrando o response 401 limpo.

```
OpenAPI env localhost:8080
1 {
2   "scheme": "http",
3   "base_path": "",
4   "host": "localhost:8080",
5   "bearerToken": "TokenInvalido"
6 }
```

Insomnia interface showing a failed POST request to `/api/veiculos`. The status is **401 Unauthorized** with a response time of 33 ms and 178 B. The response body is a JSON object:

```
{
  "path": "/api/veiculos",
  "status": 401,
  "erro": "Não autenticado",
  "timestamp": "2026-05-21T11:30:20.356123100",
  "mensagem": "É necessário realizar o login para acessar este recurso"
}
```

# Controle de acesso baseado em papéis (RBAC)

Diferenciar permissões entre analistas, administradores e usuários comuns.

O controle de acesso é implementado em duas camadas complementares. A primeira é o **SecurityConfig**, que define as regras de acesso por endpoint e método HTTP diretamente na cadeia de filtros do Spring Security, onde qualquer requisição que não atenda à regra é bloqueada antes mesmo de chegar ao controller. A segunda é o **ConsultaController**, que aplica lógica de isolamento em tempo de execução verificando o perfil do usuário autenticado via **@AuthenticationPrincipal** (ADMIN vê todas as consultas, ANALISTA vê apenas as próprias).

O sistema possui dois perfis definidos no enum **RoleUsuario**: **ADMIN** com acesso total ao sistema incluindo gerenciamento de usuários, cadastro de veículos e especificações, e visualização de todas as consultas; e **ANALISTA** com acesso restrito à leitura de veículos e especificações e visualização apenas das próprias consultas. O perfil é armazenado no token JWT no momento do login e verificado em cada requisição pelo **JwtFilter**, sem necessidade de consulta ao banco.

## Prints:

Código do **SecurityConfig.java** mostrando o bloco completo de **authorizeHttpRequests** com todas as regras de **hasRole("ADMIN")** e **hasAnyRole("ANALISTA", "ADMIN")**.

```
SecurityConfig.java x
35 public class SecurityConfig {
44     public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
64         // Regras de acesso
65         .authorizeHttpRequests( AuthorizationManagerRequestMat... auth -> auth
66
67             // Login publico
68             .requestMatchers(HttpMethod.POST, ...patterns: "/api/auth/login").permitAll()
69
70             // Swagger e H2 publicos
71             .requestMatchers(
72                 ...patterns: "/swagger-ui.html",
73                 "/swagger-ui/**",
74                 "/v3/api-docs/**",
75                 "/h2-console/**"
76             ).permitAll()
77
78             // Veículos - GET para ANALISTA e ADMIN com execucao do Listar todos (incluindo inativos) - só ADMIN
79             .requestMatchers(HttpMethod.GET, ...patterns: "/api/veiculos/todos").hasRole("ADMIN")
80             .requestMatchers(HttpMethod.GET, ...patterns: "/api/veiculos/**").hasAnyRole( ...roles: "ANALISTA", "ADMIN")
81             .requestMatchers(HttpMethod.POST, ...patterns: "/api/veiculos/**").hasRole("ADMIN")
82             .requestMatchers(HttpMethod.PUT, ...patterns: "/api/veiculos/**").hasRole("ADMIN")
83             .requestMatchers(HttpMethod.DELETE, ...patterns: "/api/veiculos/**").hasRole("ADMIN")
84             .requestMatchers(HttpMethod.PATCH, ...patterns: "/api/veiculos/**").hasRole("ADMIN")
85
86             // Especificações
87             .requestMatchers(HttpMethod.GET, ...patterns: "/api/especificacoes/**").hasAnyRole( ...roles: "ANALISTA", "ADMIN")
88             .requestMatchers(HttpMethod.POST, ...patterns: "/api/especificacoes/**").hasRole("ADMIN")
89             .requestMatchers(HttpMethod.PUT, ...patterns: "/api/especificacoes/**").hasRole("ADMIN")
90             .requestMatchers(HttpMethod.DELETE, ...patterns: "/api/especificacoes/**").hasRole("ADMIN")
91
92             // Consultas - ANALISTA e ADMIN
93             .requestMatchers( ...patterns: "/api/consultas/**").hasAnyRole( ...roles: "ANALISTA", "ADMIN")
94
95             // Usuários - só ADMIN
96             .requestMatchers( ...patterns: "/api/usuarios/**").hasRole("ADMIN")
97
98             // Qualquer outra rota exige autenticação
99             .anyRequest().authenticated()
100         )
101
102         // Registra o RateLimitFilter e o JwtFilter antes do filtro padrão do Spring
103         .addFilterBefore(rateLimitFilter, UsernamePasswordAuthenticationFilter.class)
104         .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
105
106         // Configura o provider de autenticação
107         .authenticationProvider(authenticationProvider());
108
109     return http.build();
110 }
```

Código do **ConsultaController.java** mostrando a lógica de isolamento com **@AuthenticationPrincipal** e a verificação do **role** em tempo de execução.

```
ConsultaController.java x
1 package br.com.ford.speccrader.controller;
2
3 > import ...
4
5
6
7 @RestController @ Renan
8 @RequestMapping("/api/consultas")
9 @RequiredArgsConstructor
10 @Tag(name = "Consultas", description = "Histórico de consultas realizadas")
11 @SecurityRequirement(name = "bearerAuth")
12 public class ConsultaController {
13
14     private final ConsultaService consultaService;
15
16     @Operation( @ Renan
17         summary = "Listar consultas",
18         description = "ADMIN vê todas as consultas. ANALISTA vê apenas as próprias."
19     )
20     @GetMapping
21     @ @
22     public ResponseEntity<List<ConsultaResponse>> listar(
23         @AuthenticationPrincipal Usuario usuarioLogado
24     ) {
25         if (usuarioLogado.getRole() == RoleUsuario.ADMIN) {
26             return ResponseEntity.ok(consultaService.listarTodas());
27         }
28         return ResponseEntity.ok(
29             consultaService.listarPorUsuario(usuarioLogado.getId())
30         );
31     }
32 }
```

Código do **RoleUsuario.java** mostrando o enum com os dois perfis

```
RoleUsuario.java x
1 package br.com.ford.speccrader.domain.enums;
2
3 public enum RoleUsuario { @ Renan
4     ADMIN,
5     ANALISTA
6 }
```

Teste no Insomnia logado como ANALISTA tentando criar um veículo (POST/api/veiculos) e mostrando o response 403.

POST Cadastrar veículo x +

POST .base\_url /api/veiculos Send 403 Forbidden 123 ms 169 B

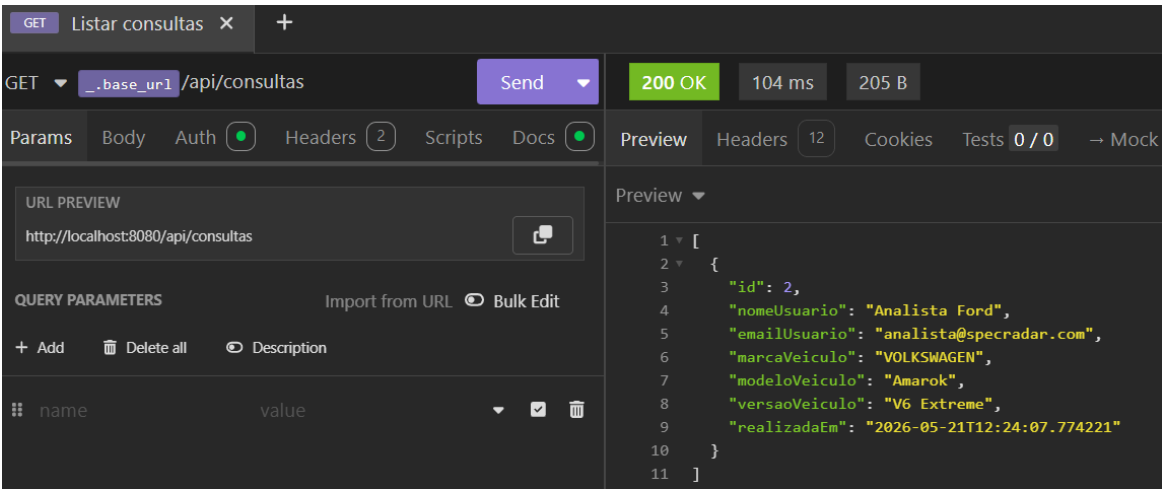
Params Body Auth Headers 3 Scripts Docs Preview Headers 9 Cookies Tests 0/0 → Mock Console

JSON Preview

```
1 {
2   "marca": "TOYOTA",
3   "modelo": "Hilux",
4   "versao": "GR-Sport",
5   "ano": 2000
6 }
```

```
1 {
2   "path": "/api/veiculos",
3   "status": 403,
4   "erro": "Acesso negado",
5   "timestamp": "2026-05-21T12:09:21.327229500",
6   "mensagem": "Você não tem permissão para acessar este recurso"
7 }
```

Teste logado como ANALISTA acessando **GET /api/consultas** e mostrando apenas as próprias consultas.



Teste logado como ADMIN acessando **GET /api/consultas** e mostrando todas as consultas do sistema incluindo as do ANALISTA.

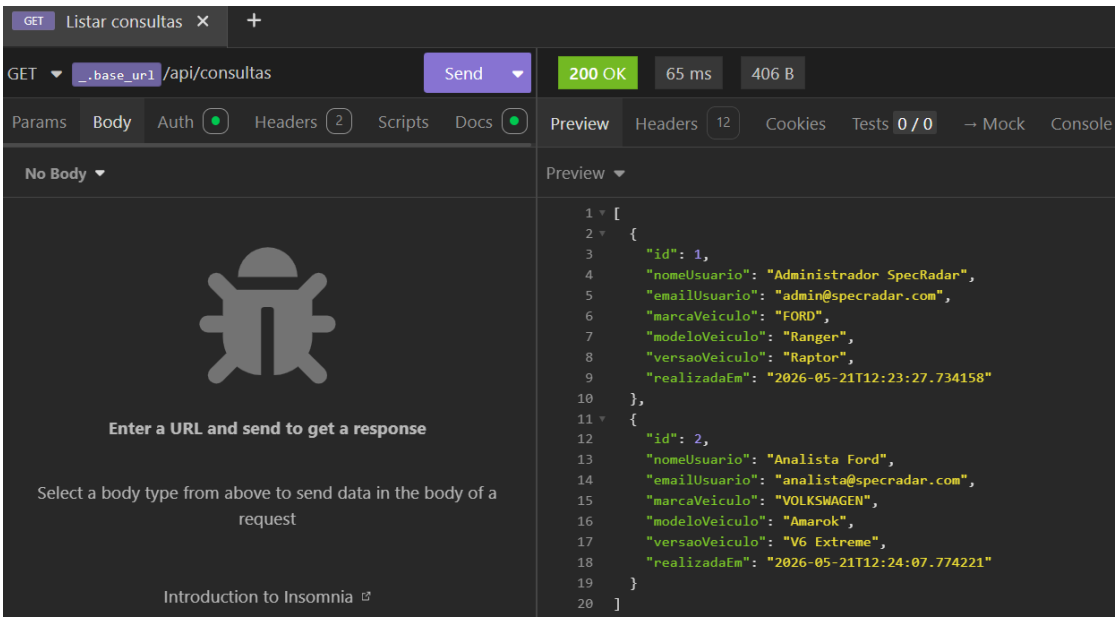


Tabela de permissões preenchida mostrando o contraste entre os dois perfis

| Perfis e Permissões (RBAC)                 |          |       |
|--------------------------------------------|----------|-------|
| Ação                                       | ANALISTA | ADMIN |
| Login                                      | ✓        | ✓     |
| Listar veículos ativos                     | ✓        | ✓     |
| Buscar veículo por ID                      | ✓        | ✓     |
| Listar especificações                      | ✓        | ✓     |
| Ver próprias consultas                     | ✓        | ✓     |
| Ver todas as consultas                     | ✗        | ✓     |
| Listar todos os veículos (inativos)        | ✗        | ✓     |
| Cadastrar / editar / desativar veículo     | ✗        | ✓     |
| Cadastrar / editar / deletar especificação | ✗        | ✓     |
| Gerenciar usuários                         | ✗        | ✓     |

### 3. Proteção de APIs e Serviços

#### Uso obrigatório de HTTPS/TLS 1.2+

Criptografia ponta a ponta entre app, API e serviços externos.

O HTTPS é configurado no perfil **prod** via certificado **PKCS12** gerado com **keytool**, a ferramenta de gerenciamento de chaves do próprio JDK. O certificado utiliza algoritmo **RSA 2048 bits** com assinatura **SHA384withRSA**, com validade de 365 dias. A separação entre perfis garante que em desenvolvimento o SSL fica desabilitado para facilitar os testes locais, enquanto em produção é ativado automaticamente apenas pela troca do **SPRING\_PROFILE**.

A porta também é definida por profile, em dev a aplicação roda em **HTTP na porta 8080**, em prod roda exclusivamente em **HTTPS na porta 8443**. Quando **server.ssl.enabled=true**, o Spring Boot desabilita o HTTP completamente, recusando qualquer acesso via **http://**. A senha do certificado nunca é hardcoded, ela é lida via variável de ambiente **SSL\_KEYSTORE\_PASSWORD** no **.env**, que está no **.gitignore**. O arquivo **specradar-ssl.p12** também está no **.gitignore** por conter material criptográfico sensível, e qualquer pessoa que precisar rodar em produção gera seu próprio certificado seguindo as instruções do README.

A configuração no **application-prod.properties** habilita o SSL com **server.ssl.enabled=true**, define a porta 8443, aponta para o keystore em **classpath:specradar-ssl.p12**, define o tipo **PKCS12** e o alias **specradar**. O Spring Boot aplica automaticamente TLS 1.2 como versão mínima. Ao acessar em prod, o navegador exibe aviso de certificado não confiável, isso é esperado para certificado self-signed e na verdade **prova que o HTTPS está funcionando**. Em produção real seria usado um certificado emitido por uma CA confiável como Let's Encrypt.

**Prints:**

**Application-prod.properties** mostrando o bloco SSL completo e a porta 8443:

```
application-prod.properties x
22 # SERVIDOR (PROD) - HTTPS
23 server.port=8443
24
25 # SSL / HTTPS
26 server.ssl.enabled=true
27 server.ssl.key-store=classpath:specradar-ssl.p12
28 server.ssl.key-store-password=${SSL_KEYSTORE_PASSWORD}
29 server.ssl.key-store-type=PKCS12
30 server.ssl.key-alias=specradar
```

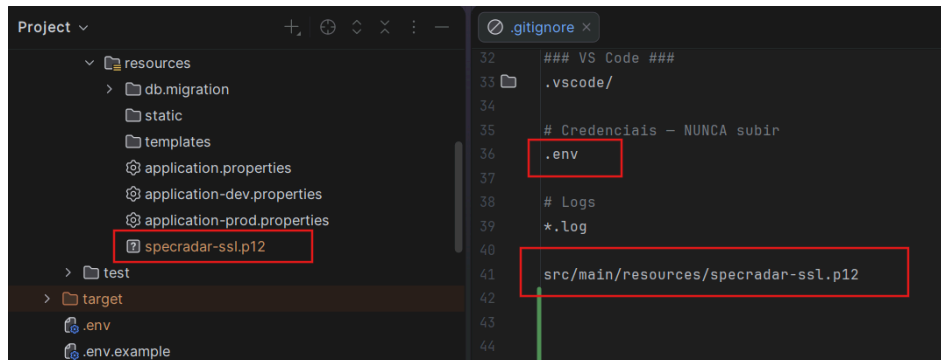
**Application-dev.properties** mostrando estar desabilitado o SSL e a porta 8080:

```
application-dev.properties x
22
23 # SERVIDOR (DEV)
24 server.port=8080
25
26 # SSL desabilitado em dev - HTTP local
27 server.ssl.enabled=false
```

Terminal mostrando o output do comando **keytool** que gerou o certificado, evidenciando o algoritmo RSA 2048 e SHA384withRSA:

```
ronal@DESKTOP-TLOIH81 MINGW64 ~/Downloads/ford-sprint-api-icers/specradar/src/main/resources (main)
$ "/c:/Program Files/Java/jdk-25.0.2/bin/keytool" -genkeypair -alias specradar -keyalg RSA -keysize 2048 -storetype PKCS12 -keystore specradar-ssl.p12 -validity 365 -storepass specradar2026 -dname "CN=specradar, OU=FIAP, O=Ford, L=Sao Paulo, ST=SP, C=BR"
Generating 2048-bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 365 days
for: CN=specradar, OU=FIAP, O=Ford, L=Sao Paulo, ST=SP, C=BR
```

Arquivo **specradar-ssl.p12** visível no explorador de arquivos do IntelliJ dentro de **src/main/resources/** (evidência que o certificado existe no projeto) e trecho do **.gitignore** mostrando as duas linhas de proteção (.env e o arquivo)



Banner de inicialização (**SpecradarApplication.java**) no console do IntelliJ rodando em **dev**:

```
=====
SpecRadar API – Ford FIAP 2026
Inteligência Competitiva Automotiva
=====
Perfil ativo : DEV
Protocolo    : HTTP
Banco       : H2 (em memória)
-----
LINKS ÚTEIS

Swagger UI : http://localhost:8080/swagger-ui.html
API Docs   : http://localhost:8080/v3/api-docs
H2 Console : http://localhost:8080/h2-console
JDBC URL   : jdbc:h2:mem:specradar
User: Ford | Password: Fiap2026
-----
CREDENCIAIS DE TESTE

ADMIN      : admin@specradar.com / Admin@2026
ANALISTA   : analista@specradar.com / Analista@2026
=====
```

Banner de inicialização (**SpecradarApplication.java**) no console do IntelliJ rodando em **prod**:

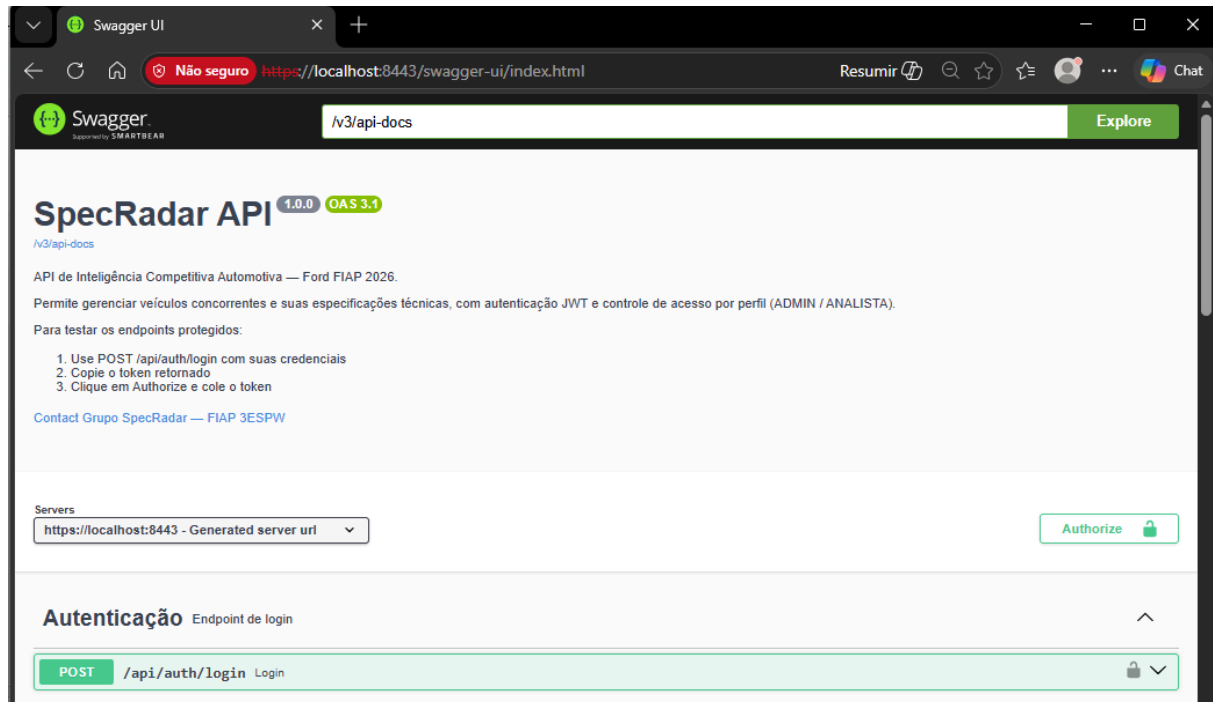
```
=====
SpecRadar API – Ford FIAP 2026
Inteligência Competitiva Automotiva
=====
Perfil ativo : PROD
Protocolo    : HTTPS
Banco       : Oracle FIAP
Usuário     : RM558540
-----
LINKS ÚTEIS

Swagger UI : https://localhost:8443/swagger-ui.html
API Docs   : https://localhost:8443/v3/api-docs

AVISO SSL: Certificado self-signed – o navegador exibirá
aviso de segurança. Clique em 'Avançar assim mesmo'.
-----
CREDENCIAIS DE TESTE

ADMIN      : admin@specradar.com / Admin@2026
ANALISTA   : analista@specradar.com / Analista@2026
=====
```

Navegador mostrando o aviso de certificado não confiável ao acessar o **Swagger UI** (<https://localhost:8443/swagger-ui.html>) em **prod**, após clicar em "Avançar assim mesmo" (esse aviso **confirma** que o HTTPS está ativo e funcionando)



## Rate limiting e throttling

Evita abuso, scraping excessivo e DoS.

O rate limiting é implementado via **RateLimitFilter** no pacote **security/**, usando a biblioteca **Bucket4j 8.10.1**. O filtro estende **OncePerRequestFilter**, garantindo que é executado exatamente uma vez por requisição. É registrado no **SecurityConfig** antes do **JwtFilter** na cadeia de filtros do Spring Security, isso significa que o bloqueio acontece antes mesmo de qualquer processamento de autenticação, protegendo inclusive o endpoint de login contra ataques de brute force.

O algoritmo utilizado é **greedy refill** com janela deslizando, onde cada IP recebe um bucket com capacidade de 20 tokens que são repostos gradualmente ao longo de 1 minuto, não todos de uma vez. Isso impede que um atacante aguarde exatamente 1 minuto e dispare 20 requisições simultâneas no início de cada janela. Cada requisição consome 1 token e ao esgotar o saldo, o IP recebe HTTP 429 com resposta JSON padronizada sem stack trace.

O filtro suporta proxies reversos via header **X-Forwarded-For**, extraindo sempre o IP real do cliente mesmo quando a requisição passa por nginx, AWS ALB ou outros proxies. O mapa de buckets por IP utiliza **ConcurrentHashMap**, garantindo thread-safety em ambientes com múltiplas requisições simultâneas. Todas as tentativas bloqueadas são logadas com prefixo **[SEGURANÇA]** incluindo IP, endpoint e método HTTP para rastreamento.

## Prints:

Trechos do **RateLimitFilter.java** mostrando:

1. A constante e o método **doFilterInternal**, onde temos a lógica de bloqueio e o log.

```
RateLimitFilter.java x
35 public class RateLimitFilter extends OncePerRequestFilter {
36     private static final Logger log = LoggerFactory.getLogger(RateLimitFilter.class); 1 usage
37
38     // Mapa de buckets por IP - cada IP tem seu próprio contador
39     private final Map<String, Bucket> buckets = new ConcurrentHashMap<>(); 1 usage
40
41     // Limite: 20 requisições por minuto por IP
42     private static final int MAX_REQUESTS_PER_MINUTE = 20; 2 usages
43
44     @Override no usages 3 Renan
45     protected void doFilterInternal(
46         @NonNull HttpServletRequest request,
47         @NonNull HttpServletResponse response,
48         @NonNull FilterChain filterChain
49     ) throws ServletException, IOException {
50
51
52         String ip = extrairIp(request);
53         Bucket bucket = buckets.computeIfAbsent(ip, this::criarNovoBucket);
54
55         // Tenta consumir 1 token - se houver saldo, deixa passar
56         if (bucket.tryConsume(numTokens: 1)) {
57             filterChain.doFilter(request, response);
58             return;
59         }
60
61         // Sem saldo - bloqueia e loga
62         log.warn("[SEGURANÇA] Rate limit excedido - IP: {} endpoint: {} método: {}",
63             ip, request.getRequestURI(), request.getMethod());
64
65         response.setStatus(HttpStatus.TOO_MANY_REQUESTS.value());
66         response.setContentType("application/json;charset=UTF-8");
67
68         Map<String, Object> body = Map.of(
69             k1: "status", v1: 429,
70             k2: "erro", v2: "Muitas requisições",
71             k3: "mensagem", v3: "Limite de requisições excedido. Tente novamente em 1 minuto.",
72             k4: "path", request.getRequestURI(),
73             k5: "timestamp", LocalDateTime.now().toString()
74         );
75
76         new ObjectMapper().writeValue(response.getOutputStream(), body);
77     }
```

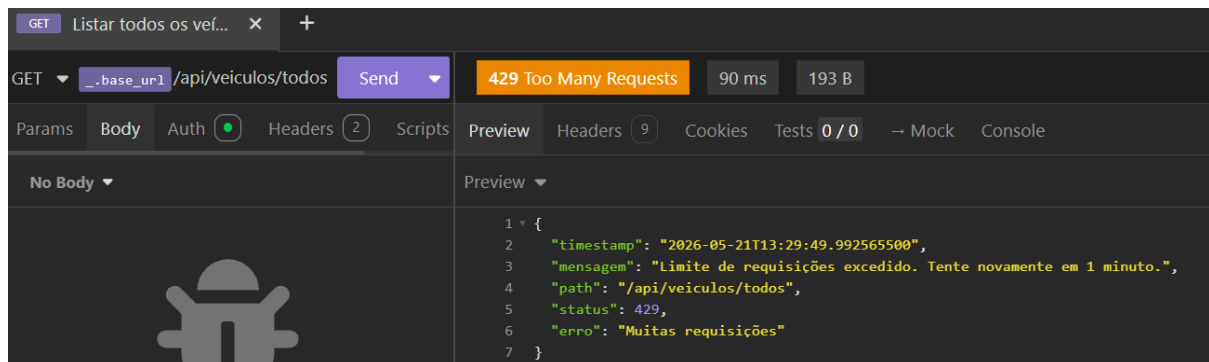
2. O método **criarNovoBucket**, onde temos o algoritmo greedy refill com Bucket4j.

```
RateLimitFilter.java x
35 public class RateLimitFilter extends OncePerRequestFilter {
79     /**
80      * Cria um novo bucket para o IP com limite de 20 req/min.
81      * Usa janela deslizante (greedy refill) - tokens são repostos
82      * gradualmente ao longo do minuto, não todos de uma vez.
83      * API atual do Bucket4j 8.x - Bandwidth.builder().
84      */
85     private Bucket criarNovoBucket(String ip) { 1 usage 2 Renan
86         Bandwidth limite = Bandwidth.builder() BandwidthBuilderCapacityStage
87             .capacity(MAX_REQUESTS_PER_MINUTE) BandwidthBuilderRefillStage
88             .refillGreedy(MAX_REQUESTS_PER_MINUTE, Duration.ofMinutes(1)) BandwidthBuilderBuildStage
89             .build();
90
91         return Bucket.builder()
92             .addLimit(limite)
93             .build();
94     }
```

Trecho do **pom.xml** mostrando a dependência do Bucket4j

```
<!-- RATE LIMITING -->
<dependency>
  <groupId>com.bucket4j</groupId>
  <artifactId>bucket4j-core</artifactId>
  <version>${bucket4j.version}</version>
</dependency>
```

Teste no Insomnia fazendo mais de 20 requisições seguidas em qualquer endpoint e mostrando o response 429



Console do IntelliJ mostrando o log gerado quando o rate limit é excedido

```
[SEGURANÇA] Rate limit excedido - IP: 127.0.0.1 endpoint: /api/veiculos/todos método: GET
```

## CORS configurado corretamente

Permitir apenas domínios autorizados.

Assinatura/verificação de integridade de payloads

Previne manipulação de dados em trânsito.

O CORS é configurado via **CorsConfig** usando **CorsFilter** do Spring, aplicado globalmente a todos os endpoints via **/\*\***. As origens permitidas são lidas exclusivamente da variável de ambiente **CORS\_ALLOWED\_ORIGINS** no **.env** e nunca há **\*** como origem permitida. Isso garante que apenas clientes autorizados, como o app mobile React Native e a interface web local, conseguem fazer requisições cross-origin à API.

Os métodos HTTP permitidos são explicitamente listados (**GET, POST, PUT, DELETE, OPTIONS**), assim como os headers aceitos (**Authorization, Content-Type, Accept, X-Requested-With**). O **allowCredentials(true)** permite o envio do header **Authorization** com o token JWT nas requisições. O tempo de cache do preflight **OPTIONS** é de 3600 segundos (1 hora), reduzindo requisições desnecessárias do browser.

A integridade dos payloads em trânsito é garantida pela assinatura **HS256** do JWT. Todo token gerado pelo **JwtService** é assinado com a chave secreta de 512 bits, onde qualquer alteração no payload do token, mesmo de um único caractere, invalida a assinatura e o token é rejeitado pelo **JwtFilter** com HTTP 401. Isso previne ataques de manipulação de dados em trânsito como MITM (Man-in-the-Middle), onde um atacante tentaria interceptar e modificar o token para elevar privilégios ou alterar a identidade do usuário.

## Prints:

Código completo do **CorsConfig.java** mostrando as origens via variável de ambiente, métodos e headers explícitos, **allowCredentials(true)** e **maxAge(3600)**

```
CorsConfig.java x
1  package br.com.ford.speccrader.config;
2
3  > import ...
13
14  @Configuration @camipzcunha
15  public class CorsConfig {
16
17      @Value("${cors.allowed-origins}")
18      private String allowedOriginsRaw;
19
20      @Bean @camipzcunha
21      public CorsFilter corsFilter() {
22          CorsConfiguration config = new CorsConfiguration();
23
24          // Origens permitidas - lidas do .env via application.properties
25          List<String> origins = Arrays.asList(allowedOriginsRaw.split(regex: " "));
26          config.setAllowedOrigins(origins);
27
28          // Métodos HTTP permitidos
29          config.setAllowedMethods(Arrays.asList(
30              "GET", "POST", "PUT", "DELETE", "OPTIONS"
31          ));
32
33          // Headers permitidos
34          config.setAllowedHeaders(Arrays.asList(
35              "Authorization",
36              "Content-Type",
37              "Accept",
38              "X-Requested-With"
39          ));
40
41          // Permite envio de credenciais (necessário para JWT no header)
42          config.setAllowCredentials(true);
43
44          // Tempo de cache do preflight (OPTIONS) em segundos - 1 hora
45          config.setMaxAge(3600L);
46
47          UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
48          source.registerCorsConfiguration(pattern: "**", config);
49
50          return new CorsFilter(source);
51      }
52  }
```

Trecho do **application.properties**

```
application.properties x
22
23 # CORS
24 cors.allowed-origins=${CORS_ALLOWED_ORIGINS}
25
```

Trecho do **.env** mostrando a variável **CORS\_ALLOWED\_ORIGINS** preenchida com as origens autorizadas

```
.env x
19
20 # CORS - origens autorizadas (app mobile Expo + web local)
21 CORS_ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8081,exp://localhost:8081
22
```

Código do **JwtService.java** mostrando o método **getSecretKey** e o **extrairTodosClaims** com **verifyWith** (evidência da verificação de assinatura em cada requisição)

```
© JwtService.java x
19 public class JwtService {
73 // Utilitários internos
74
75 private Claims extrairTodosClaims(String token) { 1 usage
76     return Jwts.parser() JwtParserBuilder
77         .verifyWith(getSecretKey())
78         .build() JwtParser
79         .parseSignedClaims(token) Jws<Claims>
80         .getPayload();
81 }
```

Teste enviando um token com payload manipulado (alterando manualmente um caractere no meio do token) e mostrando o response 401

```
OpenAPI env localhost:8080
1 {
2   "scheme": "http",
3   "base_path": "",
4   "host": "localhost:8080",
5   "bearerToken": "TokenInvalido"
6 }
```

```
POST Cadastrar veículo x +
POST _base_url /api/veiculos Send 401 Unauthorized 33 ms 178 B Just
Params Body Auth Headers 3 Scripts Docs Preview Headers 9 Cookies Tests 0/0 → Mock Console
JSON Preview
1 {
2   "marca": "TOYOTA",
3   "modelo": "Hilux",
4   "versao": "GR-Sport",
5   "ano": 2000
6 }
1 {
2   "path": "/api/veiculos",
3   "status": 401,
4   "erro": "Não autenticado",
5   "timestamp": "2026-05-21T11:30:20.356123100",
6   "mensagem": "É necessário realizar o login para acessar este recurso"
7 }
```

## 4. Segurança de Dados e Privacidade

### Criptografia de dados sensíveis em repouso

Informações de clientes, leads e histórico de manutenção.

A principal informação sensível armazenada no banco de dados é a senha dos usuários. Toda senha é criptografada com **BCrypt custo 12** antes de ser persistida, ela nunca é armazenada em texto plano em nenhum momento do fluxo, nem em logs, nem em respostas da API, nem no banco de dados. O custo 12 do BCrypt significa que o algoritmo realiza  $2^{12} = 4.096$  iterações de hashing, tornando ataques de força bruta computacionalmente inviáveis mesmo com hardware moderno.

O **PasswordEncoder** com BCrypt é configurado como bean no **SecurityConfig** e injetado no **UsuarioService**, onde toda operação que envolve senha, seja criação, atualização ou anonimização, passa obrigatoriamente pelo encoder antes de qualquer persistência. O Spring Security utiliza o mesmo encoder automaticamente na validação do login, comparando a senha fornecida com o hash armazenado via **BCrypt.checkpw** sem nunca descriptografar o hash.

Os dados de histórico de consultas armazenados na tabela **ford\_consultas** registram apenas metadados (usuário, veículo e timestamp), sem armazenar o conteúdo das consultas ou dados pessoais além do necessário. As respostas da API nunca expõem o campo **senha**. O **UsuarioResponse** omite propositalmente esse campo, garantindo que o hash BCrypt nunca trafega pela rede mesmo em endpoints administrativos.

#### Prints:

Código do **SecurityConfig.java** mostrando o bean **passwordEncoder** com **BCryptPasswordEncoder(12)**

```
SecurityConfig.java x
35  public class SecurityConfig {
169
170      @Bean  @camipzcunha
171      public PasswordEncoder passwordEncoder() {
172          return new BCryptPasswordEncoder( strength: 12);
173      }
174  }
```

Código do **UsuarioService.java** mostrando o **passwordEncoder.encode(dto.getSenha())** na criação e na atualização do usuário (a senha sempre é “encodada” antes de salvar)

```
20 public class UsuarioService {
42     @Transactional & Renan
43     @ public UsuarioResponse criar(UsuarioRequest dto) {
44         if (usuarioRepository.existsByEmail(dto.getEmail())) {
45             throw new IllegalArgumentException("Email já cadastrado: " + dto.getEmail());
46         }
47
48         Usuario usuario = Usuario.builder()
49             .nome(dto.getNome())
50             .email(dto.getEmail())
51             .senha(passwordEncoder.encode(dto.getSenha()))
52             .role(dto.getRole())
53             .ativo(true)
54             .build();
55
56         UsuarioResponse response = UsuarioResponse.fromEntity(usuarioRepository.save(usuario));
57         log.info("[AUDITORIA] Usuário criado - id: {} email: {} role: {}",
58             response.getId(), response.getEmail(), response.getRole());
59         return response;
60     }
61
62     @Transactional 3 usages & Renan
63     @ public UsuarioResponse atualizar(Long id, UsuarioRequest dto) {
64         Usuario usuario = usuarioRepository.findById(id)
65             .orElseThrow(() -> new ResourceNotFoundException("Usuario", id));
66
67         // Verifica se o novo email já pertence a outro usuário
68         if (!usuario.getEmail().equals(dto.getEmail())
69             && usuarioRepository.existsByEmail(dto.getEmail())) {
70             throw new IllegalArgumentException("Email já cadastrado: " + dto.getEmail());
71         }
72
73         usuario.setNome(dto.getNome());
74         usuario.setEmail(dto.getEmail());
75         usuario.setSenha(passwordEncoder.encode(dto.getSenha()));
76         usuario.setRole(dto.getRole());
77
78         UsuarioResponse response = UsuarioResponse.fromEntity(usuarioRepository.save(usuario));
79         log.info("[AUDITORIA] Usuário atualizado - id: {} email: {} role: {}",
80             response.getId(), response.getEmail(), response.getRole());
81         return response;
82     }
}
```

Código do **UsuarioResponse.java** mostrando que o campo **senha** está ausente (o hash nunca é exposto nas respostas)

```
1 package br.com.fond.speccrader.dto.response;
2
3 > import ...
4
11
12 @Data 28 usages & Renan
13 @Builder
14 @NoArgsConstructor
15 @AllArgsConstructor
16 public class UsuarioResponse {
17
18     private Long id;
19     private String nome;
20     private String email;
21     private RoleUsuario role;
22     private Boolean ativo;
23     private LocalDateTime criadoEm;
24
25     @ public static UsuarioResponse fromEntity(Usuario usuario) { 5 usages & Renan
26         return UsuarioResponse.builder()
27             .id(usuario.getId())
28             .nome(usuario.getNome())
29             .email(usuario.getEmail())
30             .role(usuario.getRole())
31             .ativo(usuario.getAtivo())
32             .criadoEm(usuario.getCriadoEm())
33             .build();
34     }
35 }
```

H2 Console mostrando a tabela **ford\_usuarios** com a coluna **senha** preenchida com hashes BCrypt reais

Run

Run Selected

Auto complete

Clear

SQL statement:

SELECT \* FROM FORD\_USUARIOS

SELECT \* FROM FORD\_USUARIOS;

| ID | NOME                    | EMAIL                  | SENHA                                                           | ROLE     | ATIVO | CRIADO_EM                  |
|----|-------------------------|------------------------|-----------------------------------------------------------------|----------|-------|----------------------------|
| 1  | Administrador SpecRadar | admin@specradar.com    | \$2a\$12\$qe56g7GA45wxn2vRaAsFwu0BBfMCWiBifjzWJgBs4OOdbATQIExC  | ADMIN    | 1     | 2026-05-21 12:22:24.258845 |
| 2  | Analista Ford           | analista@specradar.com | \$2a\$12\$e0TLID3z7YByNRSuAY8ZNu3pDOHzMYwISjpFYxntY4xv1fqLIUpXa | ANALISTA | 1     | 2026-05-21 12:22:24.258845 |

(2 rows, 12 ms)

Teste no Insomnia fazendo **GET /api/usuarios** como ADMIN e mostrando que o response não contém o campo **senha**

GET Listar usuários x +

GET \_base\_url /api/usuarios Send 200 OK 267 ms 277 B

Params Body Auth Headers 2 Scripts Preview Headers 12 Cookies Tests 0/0 Mock Cons

URL PREVIEW  
http://localhost:8080/api/usuarios

QUERY PARAMETERS Import from URL Bulk Edit  
+ Add Delete all Description

name value

PATH PARAMETERS

Preview

```
[
  {
    "id": 1,
    "nome": "Administrador SpecRadar",
    "email": "admin@specradar.com",
    "role": "ADMIN",
    "ativo": true,
    "criadoEm": "2026-05-21T12:22:24.258845"
  },
  {
    "id": 2,
    "nome": "Analista Ford",
    "email": "analista@specradar.com",
    "role": "ANALISTA",
    "ativo": true,
    "criadoEm": "2026-05-21T12:22:24.258845"
  }
]
```

H2 Console mostrando a tabela **ford\_consultas** com as colunas **usuario\_id**, **veiculo\_id** e **realizada\_em** (apenas metadados são armazenados, sem conteúdo sensível)

Run

Run Selected

Auto complete

Clear

SQL statement:

SELECT \* FROM FORD\_CONSULTAS

SELECT \* FROM FORD\_CONSULTAS;

| ID | USUARIO_ID | VEICULO_ID | REALIZADA_EM               |
|----|------------|------------|----------------------------|
| 1  | 1          | 4          | 2026-05-21 12:23:27.734158 |
| 2  | 2          | 2          | 2026-05-21 12:24:07.774221 |

(2 rows, 0 ms)

## Política de retenção e descarte seguro

Dados antigos devem ser removidos ou anonimizados.

A política de retenção e descarte é implementada em duas camadas complementares. A primeira é o **soft delete**, onde veículos e usuários nunca são deletados fisicamente do banco de dados, apenas marcados com **ativo = false**. Essa abordagem preserva a integridade referencial das chaves estrangeiras: se um veículo

fosse deletado fisicamente, todos os registros da tabela **ford\_consultas** vinculados a ele perderiam a referência, corrompendo o histórico de auditoria. O soft delete garante que o histórico permanece íntegro e rastreável mesmo após a desativação de um recurso.

A segunda camada é a **anonimização de dados pessoais**, implementada via endpoint exclusivo **PATCH /api/usuarios/{id}/anonimizar**, disponível apenas para ADMIN. A anonimização só pode ser aplicada em usuários já desativados, uma regra de negócio que previne a remoção acidental de dados de usuários ativos. Ao anonimizar, os dados pessoais identificáveis são sobrescritos: o nome vira "**Usuário Removido**", o email vira "**anonimizado\_{id}@specradar.com**" e a senha é substituída por um hash BCrypt de um UUID aleatório. O ID é preservado para manter as referências de chave estrangeira na tabela **ford\_consultas**, mas o registro perde qualquer dado que permita identificar a pessoa real, atendendo ao princípio de minimização de dados da **LGPD**.

Toda operação de desativação e anonimização gera log de auditoria com prefixo **[AUDITORIA]** registrando o ID e o email original antes da sobrescrita, garantindo rastreabilidade completa das operações de descarte.

### Prints:

Código do **UsuarioService.java** mostrando o método **desativar** com **ativo = false** e o log de auditoria

```
© UsuarioService.java x
20 public class UsuarioService {
84     @Transactional 3 usages 2 Renan
85     public void desativar(Long id) {
86         Usuario usuario = usuarioRepository.findById(id)
87             .orElseThrow(() -> new ResourceNotFoundException("Usuario", id));
88         usuario.setAtivo(false);
89         usuarioRepository.save(usuario);
90         log.info("[AUDITORIA] Usuário desativado - id: {} email: {}",
91             id, usuario.getEmail());
92     }
```

Código do **UsuarioService.java** mostrando o método **anonimizar** completo com a validação de usuário ativo, a sobrescrita dos dados pessoais e o log com email original

```
© UsuarioService.java x
20 public class UsuarioService {
109
110     @Transactional 4 usages 2 Renan
111     public void anonimizar(Long id) {
112         Usuario usuario = usuarioRepository.findById(id)
113             .orElseThrow(() -> new ResourceNotFoundException("Usuario", id));
114
115         if (usuario.getAtivo()) {
116             throw new IllegalArgumentException(
117                 "Apenas usuários desativados podem ser anonimizados"
118             );
119         }
120
121         String emailOriginal = usuario.getEmail();
122         usuario.setNome("Usuário Removido");
123         usuario.setEmail("anonimizado_" + id + "@specradar.com");
124         usuario.setSenha(passwordEncoder.encode(UUID.randomUUID().toString()));
125         usuarioRepository.save(usuario);
126         log.info("[AUDITORIA] Usuário anonimizado - id: {} email original: {}",
127             id, emailOriginal);
128     }
129 }
```

Código do **VeiculoService.java** mostrando o método **desativar** (Os veículos também seguem o padrão de soft delete)

```
19 public class VeiculoService {
97     @Transactional 3 usages 2 Renan
98     public void desativar(Long id) {
99         Veiculo veiculo = veiculoRepository.findById(id)
100             .orElseThrow(() -> new ResourceNotFoundException("Veiculo", id));
101         veiculo.setAtivo(false);
102         veiculoRepository.save(veiculo);
103         log.info("[AUDITORIA] Veiculo desativado - id: {} modelo: {}",
104             id, veiculo.getModelo());
105     }
}
```

Teste no Insomnia executando o fluxo completo em sequência:

## 1. DELETE /api/usuarios/3 -> 204 No Content

The first screenshot shows a DELETE request to `http://localhost:8080/api/usuarios/3` with a status of **204 No Content**. The second screenshot shows a GET request to `http://localhost:8080/api/usuarios/3` with a status of **200 OK**. The response body for the GET request is a JSON object representing the user's state after deletion.

```
1 {
2   "id": 3,
3   "nome": "Analista Teste",
4   "email": "analistaTeste@email.com",
5   "role": "ANALISTA",
6   "ativo": false,
7   "criadoEm": "2026-05-21T16:36:47.651002"
8 }
```

## 2. PATCH /api/usuarios/3/anonimizar -> 204 No Content

The first screenshot shows a PATCH request to `http://localhost:8080/api/usuarios/3/anonimizar` with a status of **204 No Content**. The second screenshot shows a GET request to `http://localhost:8080/api/usuarios/3` with a status of **200 OK**. The response body for the GET request is a JSON object representing the user's state after anonymization.

```
1 {
2   "id": 3,
3   "nome": "Usuário Removido",
4   "email": "anonimizado_3@specradar.com",
5   "role": "ANALISTA",
6   "ativo": false,
7   "criadoEm": "2026-05-21T16:36:47.651002"
8 }
```

H2 Console mostrando a tabela **ford\_usuarios** após a anonimização, evidenciando que o registro com **id=3** existe mas com dados sobrescritos

Run

Run Selected

Auto complete

Clear

SQL statement:

SELECT \* FROM FORD\_USUARIOS

SELECT \* FROM FORD\_USUARIOS;

| ID | NOME                    | EMAIL                       | SENHA                                                           | ROLE     | ATIVO | CRIADO_EM                  |
|----|-------------------------|-----------------------------|-----------------------------------------------------------------|----------|-------|----------------------------|
| 1  | Administrador SpecRadar | admin@specradar.com         | \$2a\$12\$qe56g7GA45wxn2vRaAsFwu0BBfMCWbifjzWJgBs4OOdbATQIExC   | ADMIN    | 1     | 2026-05-21 12:22:24.258845 |
| 2  | Analista Ford           | analista@specradar.com      | \$2a\$12\$e0TLiD3z7YByNRSuAY8ZNU3pDOHzMYwISJpFYxntY4xv1fqLIUpXa | ANALISTA | 1     | 2026-05-21 12:22:24.258845 |
| 3  | Usuário Removido        | anonimizado_3@specradar.com | \$2a\$12\$mE2Q80f998VsahEdCXhSMu8qV1qO1fK6i.ihvlznIMyJtnWjpuqi  | ANALISTA | 0     | 2026-05-21 16:36:47.651002 |

(3 rows, 0 ms)

Teste tentando anonimizar um usuário ainda ativo e mostrando o response 400

PTCH

Anonimizar usuário

PATCH

\_.base\_url

/api/usuarios/id/anonimizar

Send

400 Bad Request

79 ms

203 B

Params

Body

Auth

Headers

Scripts

Docs

Preview

Headers

13

Cookies

Tests

0/0

Mock

Console

URL PREVIEW

http://localhost:8080/api/usuarios/1/anonimizar

QUERY PARAMETERS

Import from URL

Bulk Edit

+ Add

Delete all

Description

name

value

Preview

1 {

2 "status": 400,

3 "erro": "Requisição inválida",

4 "mensagem": "Apenas usuários desativados podem ser anonimizados",

5 "path": "/api/usuarios/1/anonimizar",

6 "timestamp": "2026-05-21T16:50:21.5970148",

7 "campos": null

8 }

## Anonimização/pseudonimização de dados pessoais

Especialmente para modelos de ML e dashboards.

O SpecRadar implementa dois conceitos distintos e complementares de proteção de identidade, seguindo as definições da **LGPD** e boas práticas de privacidade por design.

A **anonimização** é irreversível, após executada, não existe nenhuma chave, código ou informação externa que permita recuperar os dados originais do usuário. O nome, email e senha são sobrescritos por valores fictícios que não guardam nenhuma relação com os dados originais. O registro se torna tecnicamente inútil para identificação, mas permanece no banco para preservar a integridade referencial das consultas históricas. Esse é o mecanismo adequado para dados de usuários que solicitam exclusão sob a LGPD.

A **pseudonimização** é a abordagem aplicada naturalmente ao histórico de consultas armazenado na tabela **ford\_consultas**. Os registros não armazenam nome ou email diretamente, armazenam apenas o **usuario\_id** como chave estrangeira. Isso significa que os dados de comportamento de consulta, como quais veículos foram mais consultados e em que frequência, podem ser analisados em dashboards e modelos de ML sem expor diretamente a identidade dos analistas. A identidade só pode ser recuperada cruzando com a tabela **ford\_usuarios** — operação que exige acesso privilegiado ao banco, não disponível via API para o perfil ANALISTA.

Run

Run Selected

Auto complete

Clear

SQL statement:

SELECT \* FROM FORD\_USUARIOS

SELECT \* FROM FORD\_USUARIOS;

| ID | NOME                    | EMAIL                       | SENHA                                                           | ROLE     | ATIVO | CRIADO_EM                  |
|----|-------------------------|-----------------------------|-----------------------------------------------------------------|----------|-------|----------------------------|
| 1  | Administrador SpecRadar | admin@specradar.com         | \$2a\$12\$qe56g7GA45wxn2vRaAsFwu0BBfMCWwBifljzWJgBs4OOdbATQIExC | ADMIN    | 1     | 2026-05-21 12:22:24.258845 |
| 2  | Analista Ford           | analista@specradar.com      | \$2a\$12\$e0TLID3z7YByNRSuAY8ZNu3pDOHZMYwISjPfyXntY4xv1fqLIUpXa | ANALISTA | 1     | 2026-05-21 12:22:24.258845 |
| 3  | Usuário Removido        | anonimizado_3@specradar.com | \$2a\$12\$mE2Q80f998VsaHedCXhSMu8qV1Q01K6i.hIvlnIZmYjtnWjpuqi   | ANALISTA | 0     | 2026-05-21 16:36:47.651002 |
| 4  | Usuário Removido        | anonimizado_4@specradar.com | \$2a\$12\$63NCFvx4QzQoedDmv6YZRu.Amrec158kKtrJv4IfPliCy1FYDC    | ANALISTA | 0     | 2026-05-21 20:38:56.060777 |

(4 rows, 5 ms)

## Proteção contra exposição acidental de dados

Evitar logs sensíveis, dumps e endpoints não documentados.

A proteção contra a exposição acidental é implementada em quatro frentes distintas. A primeira é a **ausência de dados sensíveis nos logs**, onde todo o projeto utiliza SLF4J com mensagens estruturadas que nunca incluem senhas, tokens JWT, hashes BCrypt ou dados pessoais completos. Os logs de auditoria registram apenas IDs e emails, nunca senhas ou tokens. O **GlobalExceptionHandler** loga internamente apenas o nome da classe da exceção e a mensagem, nunca o stack trace completo, e nunca repassa esses detalhes ao cliente.

A segunda frente é a **proteção das respostas da API**. O **UsuarioResponse** omite propositalmente o campo **senha**, garantindo que o hash BCrypt nunca trafegue pela rede em nenhum endpoint, nem mesmo nos endpoints administrativos de listagem e busca de usuários. O **spring.jpa.open-in-view=false** previne o lazy loading inesperado de entidades que poderiam expor dados não intencionais em respostas.

A terceira frente é a **proteção de credenciais e arquivos sensíveis**. O arquivo **.env** com todas as credenciais reais está no **.gitignore** e nunca vai para o repositório. O certificado SSL **specradar-ssl.p12** também está no **.gitignore**. O repositório contém apenas o **.env.example** como template sem nenhuma credencial real.

A quarta frente é a **documentação completa de todos os endpoints**, onde o Swagger documenta todos os 21 endpoints da API com descrição, request, response e códigos de erro. Não existe nenhum endpoint ativo não documentado. O **SecurityConfig** garante que qualquer rota não mapeada explicitamente exige autenticação via **.anyRequest().authenticated()**, impedindo acesso não autorizado a endpoints esquecidos.

### Prints:

Código do **UsuarioResponse.java** completo mostrando todos os campos presentes e a ausência do campo **senha** (proteção intencional)

```
UsuarioResponse.java x
1 package br.com.ford.specradar.dto.response;
2
3 > import ...
4
51
52 @Data 28 usages Renan
53 @Builder
54 @NoArgsConstructor
55 @AllArgsConstructor
56 public class UsuarioResponse {
57
58     private Long id;
59     private String nome;
60     private String email;
61     private RoleUsuario role;
62     private Boolean ativo;
63     private LocalDateTime criadoEm;
64
65 @
66 public static UsuarioResponse fromEntity(Usuario usuario) { 5 usages Renan
67     return UsuarioResponse.builder()
68         .id(usuario.getId())
69         .nome(usuario.getNome())
70         .email(usuario.getEmail())
71         .role(usuario.getRole())
72         .ativo(usuario.getAtivo())
73         .criadoEm(usuario.getCriadoEm())
74         .build();
75 }
```

## Trecho do **application.properties**

```
application.properties x
30
31 # JPA
32 spring.jpa.open-in-view=false
```

## Trecho do **.gitignore** mostrando as linhas de proteção

```
.gitignore x
35 # Credenciais - NUNCA subir
36 .env
37
38 # Logs
39 *.log
40
41 src/main/resources/specradar-ssl.p12
```

Swagger UI aberto mostrando todos os 21 endpoints documentados organizados por grupo

Swagger UI interface showing the SpecRadar API endpoints organized by group.

**Autenticação** Endpoint de login

- POST /api/auth/login Login

**Consultas** Histórico de consultas realizadas

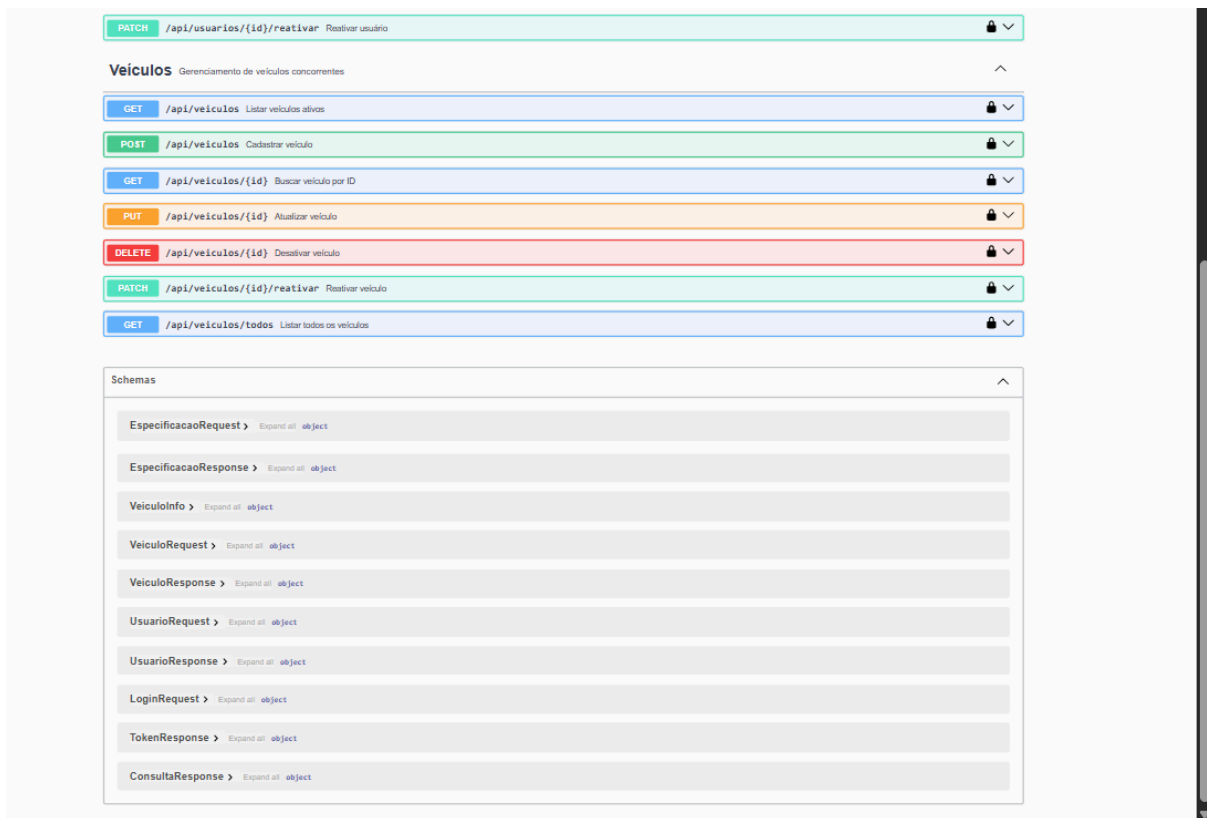
- GET /api/consultas Listar consultas

**Especificações** Gerenciamento de especificações técnicas dos veículos

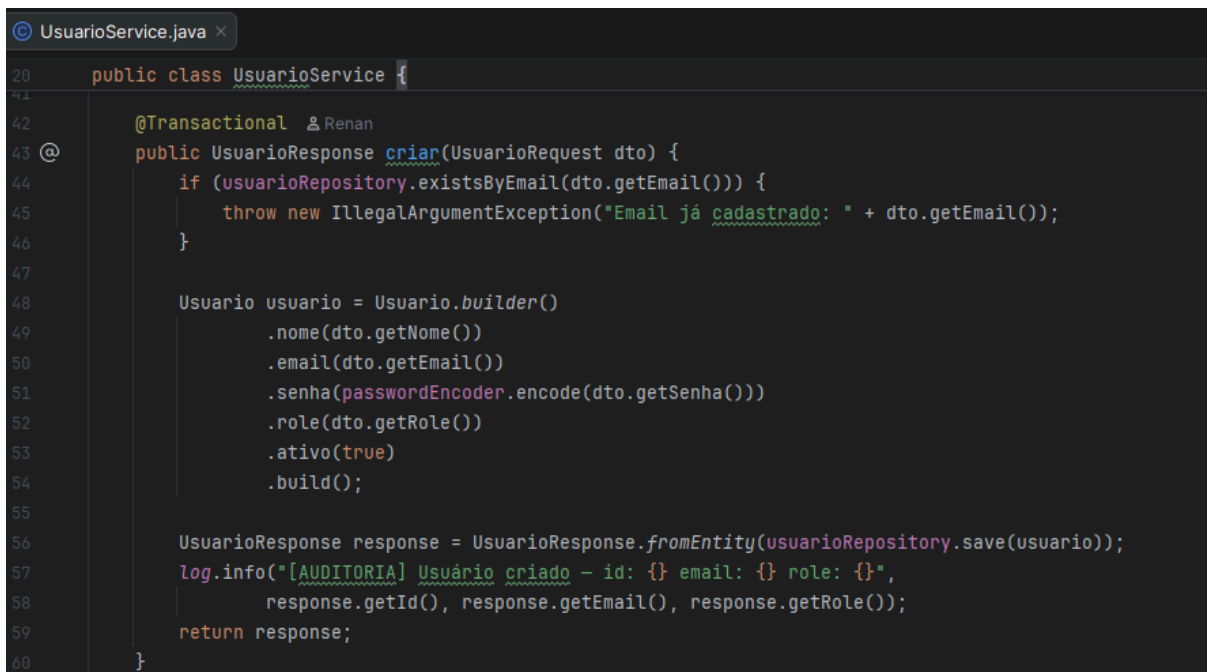
- GET /api/veiculos/{veiculoId}/especificacoes Listar especificações
- POST /api/veiculos/{veiculoId}/especificacoes Cadastrar especificação
- GET /api/veiculos/{veiculoId}/especificacoes/{specId} Buscar especificação
- PUT /api/veiculos/{veiculoId}/especificacoes/{specId} Atualizar especificação
- DELETE /api/veiculos/{veiculoId}/especificacoes/{specId} Deletar especificação

**Usuários** Gerenciamento de usuários — apenas ADMIN

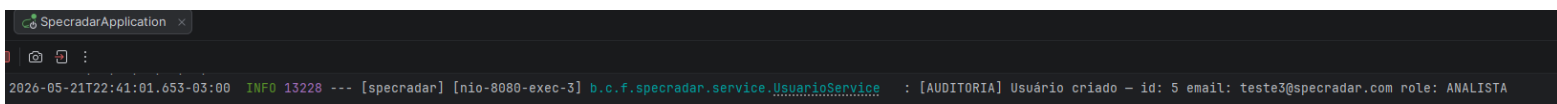
- GET /api/usuarios Listar usuários
- POST /api/usuarios Registrar usuário
- GET /api/usuarios/{id} Buscar usuário por ID
- PUT /api/usuarios/{id} Atualizar usuário
- DELETE /api/usuarios/{id} Desativar usuário
- PATCH /api/usuarios/{id}/anonimizar Anonimizar usuário



Código de **UsuarioService.java** mostrando um log de auditoria e evidenciando que nunca há senha ou token no log



Console do IntelliJ mostrando os logs gerados durante a criação de um usuário (Contêm apenas IDs, emails e roles, nunca senhas ou tokens)



## 5. Monitoramento, Logs e Auditoria

### Logs estruturados e seguros

Sem dados sensíveis, com rastreabilidade.

Todo o projeto utiliza **SLF4J** para logs, eliminando completamente o uso de **System.out.println** ou **System.err.println**. O SLF4J é o padrão da indústria para logging em aplicações Java, que permite trocar a implementação de logging subjacente sem alterar o código, e integra nativamente com o Spring Boot via Logback.

Os logs são organizados em três categorias semânticas com prefixos padronizados que facilitam a rastreabilidade e filtragem. O prefixo **[AUDITORIA]** marca todas as operações críticas (**criação, atualização, desativação, reativação e anonimização de usuários, veículos e especificações**). O prefixo **[SEGURANÇA]** marca todos os eventos de segurança (**tentativas não autenticadas, acessos negados, tokens JWT inválidos e rate limiting excedido**). O prefixo **[ERRO INTERNO]** marca exceções inesperadas capturadas pelo **GlobalExceptionHandler**, logando o nome da classe e a mensagem para diagnóstico interno sem expor ao cliente.

Nenhum log contém dados sensíveis (senhas, tokens JWT, hashes BCrypt ou dados pessoais completos nunca aparecem nos logs). Os logs de auditoria registram apenas IDs, emails e roles. Os logs de segurança registram IP, endpoint e método HTTP. Essa disciplina garante que os arquivos de log podem ser compartilhados com equipes de operação ou armazenados em sistemas de monitoramento sem risco de exposição de credenciais.

#### Prints:

Código do **UsuarioService.java** mostrando todos os **log.info("[AUDITORIA]...")** presentes em métodos (**criar, atualizar, desativar, reativar e anonimizar**) e o log gerado no Console do IntelliJ foi exibido no item anterior ("Proteção contra exposição acidental de dados").

Código do **VeiculoService.java** mostrando os **log.info("[AUDITORIA]...")** em um dos métodos (**criar, atualizar, desativar e reativar**)

```
VeiculoService.java x
19  public class VeiculoService {
97      @Transactional 3 usages 2 Renan
98      public void desativar(Long id) {
99          Veiculo veiculo = veiculoRepository.findById(id)
100             .orElseThrow(() -> new ResourceNotFoundException("Veiculo", id));
101          veiculo.setAtivo(false);
102          veiculoRepository.save(veiculo);
103          log.info("[AUDITORIA] Veiculo desativado - id: {} modelo: {}",
104             id, veiculo.getModelo());
105      }
```

Código do **EspecificacaoService.java** mostrando os **log.info("[AUDITORIA]...")** em um dos métodos (**criar, atualizar e deletar**)

```
© EspecificacaoService.java x
20 public class EspecificacaoService {
95     @Transactional 3 usages Renan
96     public void deletar(Long veiculoId, Long specId) {
97         Especificacao especificacao = especificacaoRepository.findById(specId)
98             .orElseThrow(() -> new ResourceNotFoundException("Especificacao", specId));
99
100         if (!especificacao.getVeiculo().getId().equals(veiculoId)) {
101             throw new ResourceNotFoundException("Especificacao", specId);
102         }
103
104         especificacaoRepository.delete(especificacao);
105         log.info("[AUDITORIA] Especificação deletada - id: {} veiculoId: {} atributo: {}",
106             specId, veiculoId, especificacao.getAtributo());
107     }
108 }
```

Código do **GlobalExceptionHandler.java** mostrando o log estruturado no handler genérico.

```
© GlobalExceptionHandler.java x
24 public class GlobalExceptionHandler {
147     // 500 - Erro genérico
148     @ExceptionHandler(Exception.class) 1 usage Renan
149     @
150     public ResponseEntity<ErroResponse> handleGenerico(
151         Exception ex,
152         HttpServletRequest request
153     ) {
154         log.error("[ERRO INTERNO] {} : {}", ex.getClass().getName(), ex.getMessage());
155
156         return ResponseEntity
157             .status(HttpStatus.INTERNAL_SERVER_ERROR)
158             .body(ErroResponse.of(
159                 HttpStatus.INTERNAL_SERVER_ERROR.value(),
160                 erro: "Erro interno do servidor",
161                 mensagem: "Ocorreu um erro inesperado. Tente novamente mais tarde.",
162                 request.getRequestURI()
163             ));
164     }
165 }
```

Console do IntelliJ mostrando logs de auditoria de desativação e anonimização. (Tive que cortar para a imagem sair nítida)

```
[AUDITORIA] Usuário desativado - id: 4 email: teste2@specradar.com
```

```
[AUDITORIA] Usuário anonimizado - id: 4 email original: teste2@specradar.com
```

## Monitoramento de eventos suspeitos

Tentativas de acesso indevido, falhas repetidas, anomalias.

O monitoramento de eventos suspeitos é implementado em três pontos estratégicos da cadeia de segurança, todos utilizando o prefixo **[SEGURANÇA]** nos logs para facilitar filtragem e alertas em sistemas de monitoramento externos.

O primeiro ponto é o **RateLimitFilter**, que monitora anomalias de volume de requisições. Quando um IP excede 20 requisições por minuto, o evento é logado com o IP, endpoint e método HTTP, isso permite identificar padrões de ataque como brute force no endpoint de login, scraping automatizado de especificações de veículos ou tentativas de DoS). O log é gerado antes mesmo de qualquer processamento de autenticação, garantindo rastreabilidade mesmo de requisições sem token.

O segundo ponto é o **SecurityConfig** via **AuthenticationEntryPoint** e **AccessDeniedHandler**. O **AuthenticationEntryPoint** monitora toda tentativa de acesso à endpoint protegido sem token válido, seja por ausência do token, token expirado ou token malformado. O **AccessDeniedHandler** monitora tentativas de acesso a recursos além das permissões do perfil autenticado, por exemplo, um ANALISTA tentando criar veículos, enviará o log com o IP do cliente e o endpoint acessado.

O terceiro ponto é o **JwtFilter**, que monitora tokens JWT inválidos em trânsito. Qualquer exceção durante o processamento do token (assinatura incorreta, token expirado, formato malformado) é capturada e logada com o IP, endpoint e motivo da rejeição. Isso permite identificar tentativas de forjamento de tokens ou uso de tokens de sessões anteriores já expiradas.

## Prints:

Código do **RateLimitFilter.java** mostrando o bloco de log quando o rate limit é excedido

```
RateLimitFilter.java x
35 public class RateLimitFilter extends OncePerRequestFilter {
45     @Override no usages 3 Renan
46     protected void doFilterInternal(
47         @NonNull HttpServletRequest request,
48         @NonNull HttpServletResponse response,
49         @NonNull FilterChain filterChain
50     ) throws ServletException, IOException {
51
52         String ip = extrairIp(request);
53         Bucket bucket = buckets.computeIfAbsent(ip, this::criarNovoBucket);
54
55         // Tenta consumir 1 token - se houver saldo, deixa passar
56         if (bucket.tryConsume(numTokens: 1)) {
57             filterChain.doFilter(request, response);
58             return;
59         }
60
61         // Sem saldo - bloqueia e loga
62         log.warn("[SEGURANÇA] Rate limit excedido - IP: {} endpoint: {} método: {}",
63             ip, request.getRequestURI(), request.getMethod());
64
65         response.setStatus(HttpStatus.TOO_MANY_REQUESTS.value());
66         response.setContentType("application/json;charset=UTF-8");
67
68         Map<String, Object> body = Map.of(
69             k1: "status", v1: 429,
70             k2: "erro", v2: "Muitas requisições",
71             k3: "mensagem", v3: "Limite de requisições excedido. Tente novamente em 1 minuto.",
72             k4: "path", request.getRequestURI(),
73             k5: "timestamp", LocalDateTime.now().toString()
74         );
75
76         new ObjectMapper().writeValue(response.getOutputStream(), body);
77     }
```

Código do **SecurityConfig.java** mostrando o **AuthenticationEntryPoint** com o log de tentativa não autenticada

```
SecurityConfig.java
35 public class SecurityConfig {
117     @Bean
118     public AuthenticationEntryPoint authenticationEntryPoint() {
119         return (HttpServletRequest request, HttpServletResponse response, AuthenticationException authException) -> {
120             log.warn("[SEGURANÇA] Tentativa não autenticada em {} - IP: {}",
121                 request.getRequestURI(), request.getRemoteAddr());
122
123             response.setStatus(401);
124             response.setContentType("application/json;charset=UTF-8");
125             Map<String, Object> body = Map.of(
126                 k1: "status", v1: 401,
127                 k2: "erro", v2: "Não autenticado",
128                 k3: "mensagem", v3: "É necessário realizar o login para acessar este recurso",
129                 k4: "path", request.getRequestURI(),
130                 k5: "timestamp", LocalDateTime.now().toString()
131             );
132             new ObjectMapper().writeValue(response.getOutputStream(), body);
133         };
134     }
}
```

Código do **SecurityConfig.java** mostrando o **AccessDeniedHandler** com o log de acesso negado

```
SecurityConfig.java
35 public class SecurityConfig {
136     @Bean
137     public AccessDeniedHandler accessDeniedHandler() {
138         return (HttpServletRequest request, HttpServletResponse response, AccessDeniedException accessDeniedException) -> {
139             log.warn("[SEGURANÇA] Acesso negado em {} - IP: {}",
140                 request.getRequestURI(), request.getRemoteAddr());
141
142             response.setStatus(403);
143             response.setContentType("application/json;charset=UTF-8");
144             Map<String, Object> body = Map.of(
145                 k1: "status", v1: 403,
146                 k2: "erro", v2: "Acesso negado",
147                 k3: "mensagem", v3: "Você não tem permissão para acessar este recurso",
148                 k4: "path", request.getRequestURI(),
149                 k5: "timestamp", LocalDateTime.now().toString()
150             );
151             new ObjectMapper().writeValue(response.getOutputStream(), body);
152         };
153     }
}
```

Código do **JwtFilter.java** mostrando o bloco de log de token inválido

```
JwtFilter.java
23 public class JwtFilter extends OncePerRequestFilter {
31     protected void doFilterInternal(
32         // 5. Se tem email e não há autenticação ativa no contexto
33         HttpServletRequest request, HttpServletResponse response, FilterChain filterChain) throws ServletException, IOException {
34         if (email != null && SecurityContextHolder.getContext().getAuthentication() == null) {
35
36             // 6. Carrega o usuário do banco
37             UserDetails userDetails = userDetailsService.loadUserByUsername(email);
38
39             // 7. Valida o token
40             if (jwtService.isTokenValido(token, userDetails)) {
41
42                 // 8. Cria o objeto de autenticação e injeta no contexto
43                 UsernamePasswordAuthenticationToken authToken =
44                     new UsernamePasswordAuthenticationToken(
45                         userDetails,
46                         credentials,
47                         userDetails.getAuthorities()
48                     );
49
50                 authToken.setDetails(
51                     new WebAuthenticationDetailsSource().buildDetails(request)
52                 );
53                 SecurityContextHolder.getContext().setAuthentication(authToken);
54             }
55         }
56
57         catch (Exception e) {
58             log.warn("[SEGURANÇA] Token JWT inválido na requisição {} - IP: {} - motivo: {}",
59                 request.getRequestURI(), request.getRemoteAddr(), e.getMessage());
60         }
61
62         filterChain.doFilter(request, response);
63     }
64 }
```

Console do IntelliJ mostrando log de evento suspeito gerado por excesso de requisições (Rate Limit Excedido)

```
SpecradarApplication x
2026-05-21T23:37:55.457-03:00 INFO 13228 --- [specradar] [nio-8080-exec-9] b.c.f.s.specradar.service.VeiculoService : [AUDITORIA] Veiculo cadastrado - id: 26 marca: MITSUBISHI modelo: Modelo X versao: Plus
2026-05-21T23:37:55.494-03:00 WARN 13228 --- [specradar] [nio-8080-exec-7] b.c.f.s.security.RateLimitFilter : [SEGURANÇA] Rate limit excedido - IP: 127.0.0.1 endpoint: /api/veiculos método: POST
2026-05-21T23:37:55.805-03:00 WARN 13228 --- [specradar] [nio-8080-exec-10] b.c.f.s.security.RateLimitFilter : [SEGURANÇA] Rate limit excedido - IP: 127.0.0.1 endpoint: /api/veiculos método: POST
2026-05-21T23:37:55.991-03:00 WARN 13228 --- [specradar] [nio-8080-exec-4] b.c.f.s.security.RateLimitFilter : [SEGURANÇA] Rate limit excedido - IP: 127.0.0.1 endpoint: /api/veiculos método: POST
2026-05-21T23:37:56.239-03:00 WARN 13228 --- [specradar] [nio-8080-exec-1] b.c.f.s.security.RateLimitFilter : [SEGURANÇA] Rate limit excedido - IP: 127.0.0.1 endpoint: /api/veiculos método: POST
2026-05-21T23:37:56.264-03:00 WARN 13228 --- [specradar] [nio-8080-exec-6] b.c.f.s.security.RateLimitFilter : [SEGURANÇA] Rate limit excedido - IP: 127.0.0.1 endpoint: /api/veiculos método: POST
2026-05-21T23:37:56.437-03:00 WARN 13228 --- [specradar] [nio-8080-exec-3] b.c.f.s.security.RateLimitFilter : [SEGURANÇA] Rate limit excedido - IP: 127.0.0.1 endpoint: /api/veiculos método: POST
2026-05-21T23:37:56.526-03:00 WARN 13228 --- [specradar] [nio-8080-exec-2] b.c.f.s.security.RateLimitFilter : [SEGURANÇA] Rate limit excedido - IP: 127.0.0.1 endpoint: /api/veiculos método: POST
2026-05-21T23:37:56.806-03:00 WARN 13228 --- [specradar] [nio-8080-exec-8] b.c.f.s.security.RateLimitFilter : [SEGURANÇA] Rate limit excedido - IP: 127.0.0.1 endpoint: /api/veiculos método: POST
2026-05-21T23:37:56.987-03:00 WARN 13228 --- [specradar] [nio-8080-exec-5] b.c.f.s.security.RateLimitFilter : [SEGURANÇA] Rate limit excedido - IP: 127.0.0.1 endpoint: /api/veiculos método: POST
2026-05-21T23:37:57.063-03:00 WARN 13228 --- [specradar] [nio-8080-exec-9] b.c.f.s.security.RateLimitFilter : [SEGURANÇA] Rate limit excedido - IP: 127.0.0.1 endpoint: /api/veiculos método: POST
2026-05-21T23:37:57.118-03:00 WARN 13228 --- [specradar] [nio-8080-exec-7] b.c.f.s.security.RateLimitFilter : [SEGURANÇA] Rate limit excedido - IP: 127.0.0.1 endpoint: /api/veiculos método: POST
2026-05-21T23:37:57.174-03:00 WARN 13228 --- [specradar] [nio-8080-exec-10] b.c.f.s.security.RateLimitFilter : [SEGURANÇA] Rate limit excedido - IP: 127.0.0.1 endpoint: /api/veiculos método: POST
```

## Trilha de auditoria para ações críticas

Alterações de configuração, criação de leads, consultas massivas

A trilha de auditoria do SpecRadar opera em duas camadas complementares e persistentes. A primeira é a **auditoria em banco de dados** via tabela **ford\_consultas**, onde toda vez que um analista busca um veículo por ID, o sistema registra automaticamente nos bastidores quem consultou, qual veículo e quando, sem nenhuma ação manual do usuário. Esse registro é feito pelo **ConsultaService.registrar()** chamado diretamente pelo **VeiculoController** no endpoint **GET /api/veiculos/{id}**, garantindo que nenhuma consulta passa sem rastreamento. A tabela **ford\_consultas** funciona como um ledger imutável, onde registros de consulta nunca são deletados ou modificados, apenas acumulados.

A segunda camada é a **auditoria via logs estruturados** com prefixo **[AUDITORIA]** em todos os services. Essa camada, conforme citado nos outros itens cobre as operações de escrita críticas que não têm representação física própria no banco além da alteração do registro principal (criação, atualização, desativação, reativação e anonimização de usuários, veículos e especificações).

As duas camadas se complementam: a tabela **ford\_consultas** oferece auditoria estruturada e consultável via API com isolamento por perfil, enquanto os logs **[AUDITORIA]** oferecem rastreabilidade de todas as demais operações críticas em ambientes de produção reais.

**Prints:**

Código do **VeiculoController.java** mostrando o método **buscarPorId** com a chamada automática ao **ConsultaService.registrar()**

```
VeiculoController.java x
26 public class VeiculoController {
42     @Operation(summary = "Buscar veiculo por ID", description = "Retorna um veiculo e registra a consulta") @Renan
43     @GetMapping("/{id}")
44     @PublicResponseEntity<VeiculoResponse> buscarPorId(
45         @PathVariable Long id,
46         @AuthenticationPrincipal Usuario usuarioLogado
47     ) {
48         VeiculoResponse veiculo = veiculoService.buscarPorId(id);
49
50         // Registra a consulta automaticamente nos bastidores
51         consultaService.registrar(usuarioLogado.getId(), id);
52
53         return ResponseEntity.ok(veiculo);
54     }
```

Código do **ConsultaService.java** mostrando o método **registrar** completo (a auditoria de consultas é persistida no banco)

```
19 public class ConsultaService {
20
21     @Transactional 4 usages Renan
22     public ConsultaResponse registrar(Long usuarioId, Long veiculoId) {
23         Usuario usuario = usuarioRepository.findById(usuarioId)
24             .orElseThrow(() -> new ResourceNotFoundException("Usuario", usuarioId));
25
26         Veiculo veiculo = veiculoRepository.findById(veiculoId)
27             .orElseThrow(() -> new ResourceNotFoundException("Veiculo", veiculoId));
28
29         Consulta consulta = Consulta.builder()
30             .usuario(usuario)
31             .veiculo(veiculo)
32             .build();
33
34         return ConsultaResponse.fromEntity(consultaRepository.save(consulta));
35     }
36 }
```

Teste no Insomnia fazendo **GET /api/consultas** como ADMIN mostrando que as consultas que foram efetuadas nos veículos

The screenshot shows the Insomnia API client interface. The request is a GET to `http://localhost:8080/api/consultas`. The response is a 200 OK with a status of 395 ms and 825 B. The response body is a JSON array of two consultation records:

```
1 [
2   {
3     "id": 1,
4     "nomeUsuario": "Administrador SpecRadar",
5     "emailUsuario": "admin@specradar.com",
6     "marcaVeiculo": "FORD",
7     "modeloVeiculo": "Ranger",
8     "versaoVeiculo": "Raptor",
9     "realizadaEm": "2026-05-21T12:23:27.734158"
10  },
11  {
12    "id": 2,
13    "nomeUsuario": "Analista Ford",
14    "emailUsuario": "analista@specradar.com",
15    "marcaVeiculo": "VOLKSWAGEN",
16    "modeloVeiculo": "Amarok",
17    "versaoVeiculo": "V6 Extreme",
18    "realizadaEm": "2026-05-21T12:24:07.774221"
19  },
20 ]
```

Teste no Insomnia atualizando um usuário e Console do IntelliJ mostrando o log de auditoria gerado para essa atualização

The top screenshot shows the Insomnia API client interface. The request is a PUT to `http://localhost:8080/api/usuarios/id`. The response is a 200 OK with a status of 2.3 s and 137 B. The response body is a JSON object representing the updated user:

```
1 {
2   "nome": "Teste 3 Atualizado",
3   "email": "teste3@specradar.com",
4   "senha": "TesteAtualizado",
5   "role": "ANALISTA"
6 }
```

The bottom screenshot shows the IntelliJ console output. The log entry is:

```
2026-05-22T01:12:43.780-03:00 INFO 13228 --- [specradar] [nio-8080-exec-5] b.c.f.specradar.service.UsuarioService : [AUDITORIA] Usuário atualizado - id: 5 email: teste3@specradar.com role: ANALISTA
```